FinWallet Technical and Financial Glossary

Purpose

This document is a comprehensive glossary of the main technical and financial terms used in the FinWallet architecture, technical design, source code and maintained project documents numbered 00-25.

Each term answers four questions:

- Definition: What does the term mean generally?
- FinWallet usage: Where and how is it used in this project?
- Why: Which problem does it solve?
- Note / trade-off: What can be misunderstood, what does it cost, or what alternative exists?

The glossary contains 314 terms. Terms in the "deliberately avoided" section are also included because they appear in the documentation as architecture-decision rationale.

Quick Architecture and Technical Decision Map

Choice / Technique	What is it used for in FinWallet?	Why was it chosen?
Modular Monolith	Keep the financial core in one deploy/database transaction boundary	Preserve Wallet/Ledger/Transaction atomicity and avoid premature distributed transactions
Layered + pragmatic Clean Architecture	Separate HTTP, use cases, domain rules and infrastructure	Readability, testability and technology-independent domain logic
DDD-lite	Model the financial language directly	Express Money, Wallet, Ledger and FraudDecision as real business concepts
Ports + Adapters / ACL	Isolate provider-specific contracts from Application/Domain	Allow provider changes without contaminating core rules
YARP API Gateway	Provide one public entry point with JWT, routing, rate limits and load balancing	Hide backend topology and centralize edge controls
Defense in Depth	Layer Gateway, API and internal-service validation	Reduce gateway-bypass and single-control failure risk
MSSQL	Act as the durable financial source of truth	ACID transactions, locking, constraints and audit history
Redis	Hold only transient/TTL/counter state	Provide low-latency OTP and fraud-velocity support
Double-Entry Ledger	Record every money movement as debit/credit entries	Explain source/destination of value and enforce Debit=Credit
Wallet Current Balance	Provide a fast current projection	Avoid summing the entire ledger on every request
Durable Idempotency	Prevent duplicate financial effects under retries/timeouts	Ensure the same command cannot move money twice
Outbox	Atomically store messages with committed business state	Prevent post-commit message loss after crashes
Inbox	Deduplicate provider callbacks	Prevent repeated callbacks from applying money twice
Internal + External Fraud	Combine server-side risk rules with provider risk decisions	Control transfer/purchase risk before posting
Manual Fraud Review	Pause suspicious operations before money movement	Human-in-the-loop Allow/Deny

Choice / Technique	What is it used for in FinWallet?	Why was it chosen?
Cutoff / Business Calendar	Schedule bank processing according to business days	Handle after-hours/holiday withdrawals correctly
Blocked Balance	Reserve funds during pending wallet-to-bank movement	Prevent double spending of pending funds
Compensation / Reversal / Refund	Correct committed effects using new opposite transactions	Preserve immutable audit history
Reconciliation	Compare local wallet/ledger/bank/provider records	Detect silent drift, missing callbacks and mismatches
Keyset Pagination	Read transaction history without large OFFSET scans	Stable and efficient pagination on large tables
HttpClientFactory + connection pooling	Reuse provider connections safely	Reduce socket exhaustion and latency
Structured Logging + Correlation ID	Trace distributed flows through logs	Support incident investigation and latency analysis
Docker Compose	Run the entire stack reproducibly for local/integration work	Start Gateway + APIs + MSSQL + Redis + fake providers together
Named Volumes	Keep MSSQL/Redis state independent of container recreation	Local persistence and backup convenience
GitHub Actions CI	Automate build/test/Docker/PDF validation	Catch regressions before merge
xUnit + Moq	Unit-test Application behavior quickly	Validate business branches and expected dependency interactions
Real Docker Smoke/Integration Tests	Validate DI, config, SQL schema and networking on the real stack	Catch 'builds but does not run' failures

Core Financial Concepts - Short Examples

Why are Wallet and BankAccount separate?

```
Before:
Customer FakeBank account : 5,000 TRY
FinWallet Wallet          :     0 TRY

After funding 1,000 TRY from bank to wallet:
Customer FakeBank account : 4,000 TRY
FinWallet Wallet          : 1,000 TRY
```

A Wallet is not a copy of the same row in the bank. BankAccount represents the external bank account; Wallet represents the customer's spendable balance inside FinWallet and FinWallet's obligation to that customer.

What is a Ledger?

The Ledger is the immutable accounting history that explains why money moved and between which economic accounts through debit/credit entries. The Wallet table answers "what can be spent now?"; the Ledger explains how that balance came to exist.

Example: funding 1,000 TRY from bank to wallet:

```
Debit   BANK-SETTLEMENT:TRY              1,000
Credit  WALLET-LIABILITY:<walletId>      1,000
-----------------------------------------------
Total Debit                              1,000
Total Credit                             1,000
```

Economic meaning:

- Settlement Asset +1,000: FinWallet's bank-side backing asset increased.
- Customer Wallet Liability +1,000: FinWallet now owes the customer 1,000 TRY of wallet value.

Wallet-to-Wallet Transfer example

Ali sends Ayse 300 TRY:

```
Debit   WALLET-LIABILITY:ALI              300
Credit  WALLET-LIABILITY:AYSE             300
-----------------------------------------------
Debit = Credit
```

The source liability decreases by debit and the destination liability increases by credit. No bank call is required because this is an internal book transfer.

Purchase + Platform Campaign example

Item price is 200 TRY, platform sponsors a 20 TRY discount and the customer pays 180 TRY:

```
Debit   WALLET-LIABILITY:CUSTOMER         180
Debit   CAMPAIGN-EXPENSE                   20
Credit  MERCHANT-PAYABLE                  200
-----------------------------------------------
Total Debit                               200
Total Credit                              200
```

This explicitly answers "where did the extra 20 TRY come from?": the platform funded it as an expense.

Why are Refund and Reversal not UPDATE statements?

The original transaction and journal are never deleted or overwritten. A new opposite FinancialTransaction and journal are created.

```
Original Purchase
        |
        +--> immutable history

Refund
        |
        +--> new transaction
        +--> new journal reversing the economic effect
```

This preserves the audit trail.

Why is Idempotency critical in a financial system?

The client sends a transfer, the server moves money, but the response is lost on the network. If the client retries, money must not move twice.

```
Idempotency-Key: transfer-abc-001

1st request -> operation Completed
2nd same key + same payload -> previous result replayed
same key + different payload -> Conflict
```

Why do Outbox and Inbox exist?

```
Outbox:
Financial DB COMMIT
    |
    +--> Outbox message stored in the same transaction
    |
    +--> Worker sends SMS/notification later

Inbox:
Provider callback
    |
    +--> deduplicate by Source + MessageId
    |
    +--> even 100 duplicate callbacks do not apply the financial effect twice
```

Why does Reconciliation not automatically fix balances?

Reconciliation is a discrepancy-detection mechanism, not a silent balance-overwrite mechanism.

```
Wallet current balance
        vs
Ledger-derived balance

FinWallet bank movement
        vs
FakeBank statement
```

A discrepancy creates a ReconciliationIssue. Financial history is not silently changed.

Glossary

1. Architecture and Design (32 terms)

Adapter

- Definition: A concrete implementation that connects a port to a specific technology or protocol.
- FinWallet usage: The FakeBank HTTP adapter maps IBankProvider to REST calls.
- Why / problem solved: It allows provider contracts to change without forcing Application changes.
- Note / trade-off: Validation and DTO mapping belong in the adapter; provider types should not leak into Domain.

Aggregate

- Definition: A DDD concept representing a consistency/transaction boundary around related entities and value objects.
- FinWallet usage: FinWallet does not use a heavyweight aggregate framework, but Wallet/Transaction/Ledger posting rules share one consistency boundary.
- Why / problem solved: It helps reason about which state must change atomically.
- Note / trade-off: The concept is applied lightly; not every table is treated as an aggregate.

Aggregate Root

- Definition: The main entity through which an aggregate is accessed.
- FinWallet usage: FinWallet does not enforce a heavy aggregate-root framework, but Wallet behavior is protected around its identity and invariants.
- Why / problem solved: It provides controlled entry to aggregate consistency rules.
- Note / trade-off: The term is used lightly because the project has no formal aggregate-root hierarchy.

Anti-Corruption Layer (ACL)

- Definition: A translation layer that isolates an external system's model from the internal domain model.
- FinWallet usage: Bank, Fraud, Cutoff and Campaign responses are converted into FinWallet types inside Infrastructure.
- Why / problem solved: It prevents external naming, enums and error semantics from corrupting the domain.
- Note / trade-off: It adds mapping code but substantially reduces integration coupling.

Append-Only

- Definition: A data model where new history records are appended and prior records are not overwritten or deleted.

- FinWallet usage: Ledger journals/entries and correction history follow this principle.
- Why / problem solved: It preserves auditability and prevents rewriting financial history.
- Note / trade-off: Current-state tables do not need to be append-only; history and projection serve different purposes.

Audit Trail

- Definition: A record allowing the history of business/financial state changes to be traced back to events and actors.
- FinWallet usage: FinancialTransaction, Ledger, FraudEvent, reconciliation and review metadata together form an audit trail.
- Why / problem solved: It supports incident, dispute and financial investigation.
- Note / trade-off: Audit records are not the same as debug logs and may require stronger retention/integrity controls.

Bounded Context

- Definition: A business boundary within which terms have a consistent meaning.
- FinWallet usage: FinWallet's financial core and FakeBank/FakeFraud/FakeCampaign provider models are treated as separate semantic boundaries.
- Why / problem solved: It prevents provider DTOs from leaking directly into the domain model.
- Note / trade-off: In v1 these are conceptual boundaries protected by adapters, not a full microservice bounded-context organization.

Clean Architecture

- Definition: A set of architectural principles where dependencies point inward toward business rules.
- FinWallet usage: Domain knows no SQL/HTTP/framework details; Application defines ports and Infrastructure implements them.
- Why / problem solved: It keeps business rules technology-independent and improves testability.
- Note / trade-off: FinWallet is a pragmatic simplified Clean Architecture, not a maximal academic implementation.

Consistency Boundary

- Definition: The boundary of data that must remain correct together during an operation.
- FinWallet usage: Wallet balance, FinancialTransaction, Ledger, durable idempotency and sometimes Outbox are committed in one MSSQL transaction.
- Why / problem solved: It prevents partial financial commits.
- Note / trade-off: External provider state lies outside this boundary, which is why compensation and reconciliation are needed.

DDD-lite

- Definition: A pragmatic use of useful DDD modeling ideas without heavyweight process or infrastructure.
- FinWallet usage: FinWallet uses value objects, entities, invariants and domain language but not full Event Sourcing or a large domain-event bus.
- Why / problem solved: It keeps the financial domain expressive while limiting over-engineering.
- Note / trade-off: More complex future domains may require stronger DDD mechanisms.

Defense in Depth

- Definition: Applying multiple layers of security controls instead of relying on a single control.
- FinWallet usage: Gateway validates JWT while FinWallet.Api independently validates JWT/session/ownership and service credentials.
- Why / problem solved: If one layer is bypassed, another can still reject the request.
- Note / trade-off: It is not blind duplication; each layer protects a different trust boundary.

Dependency Injection (DI)

- Definition: Providing a class's dependencies from the outside.
- FinWallet usage: ASP.NET Core built-in DI registers handlers, stores, providers, TimeProvider and background services.
- Why / problem solved: It removes object construction from business classes and improves testability/runtime composition.
- Note / trade-off: Missing registrations may fail only at startup or first resolution, so runtime smoke testing is important.

Dependency Inversion Principle (DIP)

- Definition: The principle that high-level business logic depends on abstractions rather than low-level details.
- FinWallet usage: Application depends on interfaces such as IBankProvider, stores and fraud-provider ports; Infrastructure supplies implementations.
- Why / problem solved: It makes provider substitution and unit-test mocking easier.
- Note / trade-off: It does not mean creating an interface for every class; FinWallet uses it at real boundaries.

Dependency Rule

- Definition: The rule that inner layers must not depend on outer layers.
- FinWallet usage: FinWallet keeps Api -> Application -> Domain and Infrastructure -> Application/Domain dependency directions.
- Why / problem solved: It prevents Domain from depending on SQL, Redis or provider DTOs.
- Note / trade-off: Breaking the rule couples core business logic to technology choices.

Domain-Driven Design (DDD)

- Definition: An approach that models software around real business concepts and rules.
- FinWallet usage: Wallet, Money, Currency, FinancialTransaction, Ledger and FraudDecision are direct domain concepts.
- Why / problem solved: It reduces semantic drift between code and the finance/banking domain.
- Note / trade-off: FinWallet uses DDD-lite; it avoids heavyweight aggregate/event-sourcing infrastructure.

Entity

- Definition: A domain object with identity that can change state over time.
- FinWallet usage: Wallet, BankAccount and FinancialTransaction are identity-based entities.
- Why / problem solved: It keeps lifecycle and rules for the same business identity together.

- Note / trade-off: Entity equality is identity-oriented, not just value-oriented.

Handler

- Definition: An Application class that orchestrates a use-case command or query.
- FinWallet usage: Classes such as ExecuteWalletTransferHandler and ExecuteFraudProtectedPurchaseHandler serve this role.
- Why / problem solved: It keeps controllers thin and separates business flow from HTTP.
- Note / trade-off: Handlers are invoked directly through DI; MediatR is not used.

Horizontal Scaling

- Definition: Increasing capacity by running more instances/replicas of the same service.
- FinWallet usage: YARP clusters can contain multiple FinWallet.Api or provider replicas.
- Why / problem solved: It reduces single-instance bottlenecks.
- Note / trade-off: Correctness must rely on DB constraints, idempotency and concurrency rules rather than instance-local state.

Invariant

- Definition: A business rule that must remain true regardless of execution path.
- FinWallet usage: Debit == Credit, no negative wallet balance and no double correction are invariants.
- Why / problem solved: It makes financial correctness a model/data property rather than an endpoint convention.
- Note / trade-off: It is not a runtime toggle; changing an invariant may require data/accounting migration.

Layered Architecture

- Definition: An architecture that separates responsibilities into layers.
- FinWallet usage: FinWallet uses separate Api, Application, Domain and Infrastructure projects.
- Why / problem solved: It prevents HTTP, use-case, domain-rule and infrastructure concerns from becoming mixed together.
- Note / trade-off: Too many abstractions can cause over-engineering; the project intentionally keeps the layers small.

Modular Monolith

- Definition: An architectural style where domain areas are separated into modules inside one deployable application.
- FinWallet usage: FinWallet keeps Wallet, BankAccount, Transaction, Ledger, Fraud and Reconciliation modules inside the same application and MSSQL transaction boundary.
- Why / problem solved: It reduces the need for distributed transactions while preserving domain separation and atomic financial commits.
- Note / trade-off: It does not provide microservice-level independent deployment; v1 prioritizes financial correctness and simplicity.

Orchestration

- Definition: Coordinating the ordered steps of a use case.

- FinWallet usage: The transfer handler manages replay -> risk signals -> internal fraud -> external fraud -> durable fraud -> posting.
- Why / problem solved: It is critical for keeping external I/O and DB transaction boundaries correct.
- Note / trade-off: The orchestrator should coordinate, not absorb every business rule itself.

Port

- Definition: An interface/boundary defined by Application to interact with the outside world.
- FinWallet usage: IBankProvider, IExternalFraudProvider and store interfaces are ports.
- Why / problem solved: It describes the capability Application needs without choosing a technology.
- Note / trade-off: The implementation lives in Infrastructure.

Projection

- Definition: A current read-friendly view derived from history or business state.
- FinWallet usage: Wallet AvailableBalance/BlockedBalance can be viewed as current state reconciled against ledger history.
- Why / problem solved: It avoids summing the entire ledger on every request.
- Note / trade-off: A projection can drift from its source history, which is why reconciliation is required.

Separation of Concerns

- Definition: The principle of keeping different responsibilities out of the same class or layer.
- FinWallet usage: HTTP mapping lives in Controllers, orchestration in Application, rules in Domain and SQL/HTTP clients in Infrastructure.
- Why / problem solved: It reduces change impact and improves readability.
- Note / trade-off: Over-fragmentation is also costly; FinWallet uses relatively coarse practical boundaries.

State Machine / Lifecycle

- Definition: A model defining allowed states and transitions for an entity or transaction.
- FinWallet usage: FinancialTransaction and bank-movement flows use states such as Scheduled, Pending, Completed and Failed.
- Why / problem solved: It prevents invalid transitions and makes retry/callback behavior deterministic.
- Note / trade-off: State names represent domain lifecycle, not merely API response labels.

Stateless Service

- Definition: A service that does not keep durable business truth in process/container memory.
- FinWallet usage: Gateway, FinWallet.Api and fake HTTP services can be recreated; durable financial state lives in MSSQL.
- Why / problem solved: It enables safer horizontal scaling and container restarts.
- Note / trade-off: Redis/cache may hold transient state, but financial truth is not tied to process memory.

Topology

- Definition: The runtime arrangement of services, network paths, data stores and trust boundaries.

- FinWallet usage: 25-topology.md describes Gateway, API, fake providers, MSSQL, Redis, workers and Docker networks.
- Why / problem solved: It explains actual traffic/dependency flow rather than only source-code structure.
- Note / trade-off: Development and production topologies may differ in replicas and edge infrastructure.

Transaction Boundary

- Definition: The design decision defining where a database transaction starts and ends.
- FinWallet usage: FinWallet never keeps a SQL transaction open during external HTTP; it opens a short transaction after the provider result.
- Why / problem solved: This reduces lock duration, deadlock risk and connection-pool pressure.
- Note / trade-off: It does not provide distributed atomicity; external mismatches are handled through reconciliation.

Trust Boundary

- Definition: A security boundary where data or identity crossing from one side is not automatically trusted.
- FinWallet usage: Client->Gateway, Gateway->FinWallet.Api, FinWallet.Api->Gateway provider route and Gateway->provider are separate trust boundaries.
- Why / problem solved: It reduces risks such as gateway bypass and credential reuse.
- Note / trade-off: Each boundary must perform its own validation.

Use Case

- Definition: An application behavior that fulfills one user or system goal.
- FinWallet usage: Register, Login, CreateWallet, BankDeposit, Transfer, Purchase, Refund and Reconciliation are use cases.
- Why / problem solved: It centralizes orchestration between controllers and domain/persistence details.
- Note / trade-off: A use case does not have to map one-to-one to an HTTP endpoint.

Value Object

- Definition: An immutable domain object whose value and validation rules matter more than identity.
- FinWallet usage: Money and currency concepts are modeled this way.
- Why / problem solved: It keeps amount/currency together and prevents invalid monetary values from entering the domain.
- Note / trade-off: Value objects should remain small and immutable.

2. API, Gateway and Networking (32 terms)

Active Health Check

- Definition: A proxy-driven periodic probe of backend destinations.
- FinWallet usage: YARP can actively check FinWallet/FakeBank/FakeFraud destinations.
- Why / problem solved: It detects failures even when no user traffic is flowing.
- Note / trade-off: Excessive probing adds load, so intervals/timeouts are configurable.

API (Application Programming Interface)

- Definition: A contract defining how software can be invoked by other software.
- FinWallet usage: FinWallet exposes public customer APIs and internal provider APIs over HTTP.
- Why / problem solved: It provides a stable contract between clients and business logic.
- Note / trade-off: The API contract should not mirror the domain model directly; DTOs and ServiceResult preserve the boundary.

API Gateway

- Definition: An edge application that provides a shared entry point to backend services.
- FinWallet usage: FinWallet.Gateway performs JWT validation, routing, rate limits, service-key policies and load balancing.
- Why / problem solved: It hides backend topology from clients and centralizes shared edge policies.
- Note / trade-off: Domain business logic does not belong in the Gateway.

Cluster

- Definition: A YARP group containing one or more destinations for the same logical backend.
- FinWallet usage: FinWallet.Api and each fake provider are configured as separate clusters.
- Why / problem solved: It enables replicas, load balancing and health policies.
- Note / trade-off: A single development destination is still useful because the cluster model supports production scaling.

Connection Keep-Alive

- Definition: Reusing the same TCP/HTTP connection for multiple requests.
- FinWallet usage: FinWallet controls it through Kestrel and outbound HttpClient lifetime settings.
- Why / problem solved: It reduces handshake cost and latency.
- Note / trade-off: Excessively long lifetimes can cause stale connections or delayed DNS changes.

Content-Type

- Definition: An HTTP header describing the format of the request body.
- FinWallet usage: FinWallet requires application/json for write requests.
- Why / problem solved: It reduces ambiguous or unexpected payload parsing.
- Note / trade-off: The requirement does not apply to requests without a body.

Controller

- Definition: An ASP.NET Core class that receives HTTP requests and delegates to Application use cases.
- FinWallet usage: FinWallet uses controller-based APIs rather than Minimal APIs.
- Why / problem solved: It separates HTTP concerns from business orchestration and provides clear Swagger metadata.
- Note / trade-off: Business logic should not accumulate inside controllers.

Correlation ID

- Definition: A unique reference propagated across services to trace one request flow.

- FinWallet usage: X-Correlation-Id travels through Gateway, API and provider calls and is regenerated if invalid.
- Why / problem solved: It correlates distributed logs for the same operation.
- Note / trade-off: It is not an idempotency key or FinancialTransactionId and provides no duplicate protection.

CORS

- Definition: A browser policy controlling which cross-origin requests are permitted.
- FinWallet usage: Allowed origins are configured as an appsettings allow-list.
- Why / problem solved: It limits browser-based cross-origin access.
- Note / trade-off: CORS is not authentication and does not block non-browser clients.

Destination

- Definition: A concrete backend instance address inside a YARP cluster.
- FinWallet usage: For example, http://finwallet-api:8080 can be a Docker destination.
- Why / problem solved: It is the actual service instance selected by the load balancer.
- Note / trade-off: An unhealthy destination can be removed from traffic based on health checks.

DTO (Data Transfer Object)

- Definition: An object used to transfer data across layers or system boundaries.
- FinWallet usage: Request/response models and FakeBank/FakeFraud provider contracts are DTOs.
- Why / problem solved: It prevents direct exposure of domain entities.
- Note / trade-off: DTOs add validation/mapping work but make boundaries explicit.

Endpoint

- Definition: An API operation addressed by a specific HTTP method and URL combination.
- FinWallet usage: POST /api/v1/transfers and GET /api/v1/transactions are examples.
- Why / problem solved: It is the external entry point to a use case.
- Note / trade-off: Endpoints should map requests/responses rather than contain core business logic.

Fixed-Window Rate Limiter

- Definition: A rate-limit algorithm that divides time into fixed windows with a permit count per window.
- FinWallet usage: Shared.Web uses this model for the global limiter.
- Why / problem solved: It is simple, inexpensive and configuration-driven.
- Note / trade-off: Bursts can occur at window boundaries; sliding-window or token-bucket models may suit stricter needs.

Health Check

- Definition: A mechanism that determines whether a service is fit to receive traffic.
- FinWallet usage: Gateway/Compose use health endpoints and provider health checks.
- Why / problem solved: It reduces traffic to broken instances and validates startup dependencies.
- Note / trade-off: Health design should distinguish process liveness from dependency readiness when appropriate.

HTTP Status Code

- Definition: A standardized numeric result of an HTTP response.
- FinWallet usage: FinWallet uses statuses such as 200, 202, 400, 401, 403, 404, 409, 422, 429 and 503.
- Why / problem solved: It standardizes transport-level outcome while machine codes provide business detail.
- Note / trade-off: The same HTTP status can contain different business codes; clients should not rely on status alone.

HttpClientFactory

- Definition: .NET infrastructure for centrally managing HttpClient lifetimes and handler pooling.
- FinWallet usage: Bank, Fraud, Cutoff, Campaign and Communication typed clients are created through it.
- Why / problem solved: It reduces socket exhaustion and incorrect HttpClient lifetime patterns.
- Note / trade-off: It does not automatically make financial POST retries safe; retry semantics are designed separately.

JSON

- Definition: A text-based data format commonly used by HTTP APIs.
- FinWallet usage: FinWallet requires application/json for write requests.
- Why / problem solved: It provides easy cross-platform serialization and debugging.
- Note / trade-off: Financial decimals and date formats still need controlled contract semantics.

Kestrel

- Definition: ASP.NET Core's HTTP server.
- FinWallet usage: Shared.Web configures request-body/header, connection, keep-alive and header-timeout limits through Kestrel.
- Why / problem solved: It bounds resource consumption and header/body abuse at the application level.
- Note / trade-off: It does not replace edge protection and can run behind a reverse proxy/load balancer.

Load Balancing

- Definition: Distributing requests across multiple backend instances.
- FinWallet usage: YARP can distribute production traffic across replicas in the same cluster.
- Why / problem solved: It improves capacity and availability.
- Note / trade-off: Financial correctness must not depend on sticky sessions; durable state belongs in MSSQL.

OpenAPI

- Definition: A machine-readable specification for describing REST API contracts.
- FinWallet usage: Swagger tooling produces an OpenAPI document behind the scenes, while Swagger is the user-facing term in the project.
- Why / problem solved: It enables tooling, client generation and schema discovery.

- Note / trade-off: FinWallet's domain architecture does not depend on OpenAPI.

Passive Health Check

- Definition: Health evaluation based on failures observed during real traffic.
- FinWallet usage: A FinWallet API destination may be temporarily deactivated after transport failures.
- Why / problem solved: It uses real request failures without extra probes.
- Note / trade-off: Low traffic can delay detection, so it is stronger when combined with active checks.

PowerOfTwoChoices

- Definition: A load-balancing algorithm that samples two candidates and selects the less-loaded one.
- FinWallet usage: It is the preferred YARP policy for FinWallet clusters.
- Why / problem solved: It can provide better load distribution than simple round-robin with little overhead.
- Note / trade-off: With one destination there is no meaningful choice; it matters when replicas are added.

Rate Limiting

- Definition: Limiting how many requests are accepted within a period.
- FinWallet usage: Gateway applies per-IP fixed-window limits and backend can retain a defense-in-depth limit.
- Why / problem solved: It reduces resource exhaustion, brute force and simple L7 abuse.
- Note / trade-off: It is not volumetric DDoS protection; edge DDoS/WAF controls are still required.

REST

- Definition: A web-API style that uses HTTP resource and verb semantics.
- FinWallet usage: FinWallet controller endpoints use HTTP methods such as GET/POST with JSON payloads.
- Why / problem solved: It provides a simple, widely supported integration model.
- Note / trade-off: FinWallet uses pragmatic REST rather than a strict HATEOAS-oriented academic implementation.

Retry-After

- Definition: An HTTP response header telling the client when it may retry.
- FinWallet usage: FinWallet adds it to rate-limit rejections when available.
- Why / problem solved: It reduces immediate repeated retries from clients.
- Note / trade-off: Retries of financial POSTs still depend on idempotency and provider semantics.

Reverse Proxy

- Definition: An HTTP intermediary that receives a request and forwards it to a backend service.
- FinWallet usage: Gateway forwards public /api/* traffic to FinWallet.Api and /providers/* traffic to fake providers.
- Why / problem solved: It hides backend addresses and centralizes traffic controls.
- Note / trade-off: The proxy must not become a business source of truth; financial state is not stored in Gateway.

Route

- Definition: A rule determining which backend cluster receives an incoming path.
- FinWallet usage: FinWallet has separate YARP routes for public auth, protected APIs, internal callbacks and /providers/*.
- Why / problem solved: It controls path-level routing and authorization policy selection.
- Note / trade-off: Incorrect precedence can route internal endpoints through the wrong authorization policy.

ServiceResult<T>

- Definition: FinWallet's common envelope for consistent success/failure responses.
- FinWallet usage: It carries fields such as isSuccess, code, message, data and errors across APIs.
- Why / problem solved: It prevents clients from parsing a different error shape for every endpoint.
- Note / trade-off: It does not replace HTTP status; transport status and ServiceResult code are used together.

SocketsHttpHandler

- Definition: The low-level .NET HTTP handler controlling connection pools, DNS/lifetimes and transport behavior.
- FinWallet usage: FinWallet configures PooledConnectionLifetime, idle timeout and MaxConnectionsPerServer.
- Why / problem solved: It balances connection reuse with DNS refresh in long-running services.
- Note / trade-off: Values should not be increased aggressively without measurement.

Swagger

- Definition: An interactive interface for discovering API endpoints, request/response schemas and contracts.
- FinWallet usage: Gateway, FinWallet.Api and all fake APIs expose it through Shared.Web/Swashbuckle.
- Why / problem solved: It improves developer onboarding, manual testing and contract visibility.
- Note / trade-off: It is disabled by default in production and never bypasses endpoint authorization.

Timeout

- Definition: A time limit preventing an I/O operation from waiting indefinitely.
- FinWallet usage: Provider HttpClients, YARP activity and request-header processing are bounded by configurable timeouts.
- Why / problem solved: It reduces blocked resources and cascading stalls.
- Note / trade-off: Timeouts that are too short cause false failures; too long can exhaust resources.

YARP

- Definition: Microsoft's reverse-proxy toolkit for .NET.
- FinWallet usage: FinWallet.Gateway uses YARP for routes, clusters, load balancing, health and request transforms.
- Why / problem solved: It provides a configuration-driven gateway without custom low-level proxy code.

- Note / trade-off: YARP is an application gateway; it is not a substitute for volumetric DDoS edge/WAF protection.

3. Security and Identity (39 terms)

sid Claim

- Definition: A JWT claim carrying the session identifier.
- FinWallet usage: The durable session ID created at login is included in the access token and used for logout and fraud signals.
- Why / problem solved: It links the token to server-side session lifecycle and revocation state.
- Note / trade-off: This adds session validation beyond cryptographic token validation alone.

sub Claim

- Definition: The standard JWT subject claim.
- FinWallet usage: In FinWallet it carries the authenticated CustomerId.
- Why / problem solved: It allows controllers to derive owner context without trusting a customer ID from the request body.
- Note / trade-off: An invalid or missing GUID leads to INVALID_ACCESS_TOKEN.

Access Token

- Definition: A short-lived credential used to call protected APIs.
- FinWallet usage: FinWallet issues it as a JWT after login or refresh.
- Why / problem solved: It authenticates user identity on each request.
- Note / trade-off: Short lifetimes reduce stolen-token risk but require a refresh mechanism.

Authentication

- Definition: Verifying who a user or service is.
- FinWallet usage: Public clients use login/JWT while internal service calls use service keys.
- Why / problem solved: It prevents unauthenticated actors from reaching protected endpoints.
- Note / trade-off: Authentication does not decide what an identity may do; authorization is a separate concern.

Authorization

- Definition: Checking whether an authenticated identity is allowed to perform an action on a resource.
- FinWallet usage: FinWallet uses gateway policies, [Authorize], owner-aware SQL and internal-service policies.
- Why / problem solved: It reduces risks such as accessing another customer's wallet or transaction data.
- Note / trade-off: Role checks alone are insufficient; resource ownership is also validated.

Bearer Token

- Definition: A token usage model where possession of the token allows it to be presented in the HTTP Authorization header.
- FinWallet usage: Authorization: Bearer <JWT> is used on protected client endpoints.

- Why / problem solved: It integrates cleanly with standard HTTP tooling.
- Note / trade-off: A stolen token remains risky until expiry or session revocation, so TLS and short lifetimes are important.

BOLA (Broken Object Level Authorization)

- Definition: A vulnerability where an authenticated user accesses another user's object by changing an identifier.
- FinWallet usage: FinWallet validates customer ownership for wallets, transactions and bank accounts on the server side.
- Why / problem solved: It prevents cross-customer access through guessed or manipulated IDs.
- Note / trade-off: JWT validation alone does not solve BOLA; object-level ownership checks are required.

CDN

- Definition: A distributed network that serves/caches content from edge locations.
- FinWallet usage: It is not a required FinWallet API component but can appear in production edge/DDoS architecture discussions.
- Why / problem solved: It can provide static-content delivery and edge traffic absorption.
- Note / trade-off: Caching authenticated financial API responses requires extreme care.

Claim

- Definition: A name/value statement carried inside a JWT about identity or session context.
- FinWallet usage: sub identifies the customer and sid identifies the session.
- Why / problem solved: Claims let Gateway/API associate requests with server-issued identity.
- Note / trade-off: Using claims instead of client-supplied customer IDs reduces spoofing.

CSP (Content Security Policy)

- Definition: A browser response policy limiting allowed script, style, frame and other content sources.
- FinWallet usage: Shared.Web applies a restrictive CSP to APIs and a Swagger-compatible CSP to Swagger pages.
- Why / problem solved: It is a browser defense-in-depth control against XSS/frame injection.
- Note / trade-off: Its impact on pure APIs is limited, but it provides a secure default.

Data Masking

- Definition: Hiding all or part of sensitive values in logs or responses.
- FinWallet usage: Tokens, OTPs, passwords and secrets are not logged; PII is minimized or masked when needed.
- Why / problem solved: It balances operational visibility with privacy/security.
- Note / trade-off: Masking is not encryption; the underlying sensitive data still requires protection.

DDoS Protection

- Definition: Infrastructure that protects network/edge capacity against very high-volume distributed traffic attacks.
- FinWallet usage: Application rate limits only address L7 abuse; production needs cloud/load-balancer/CDN DDoS protection.

- Why / problem solved: Volumetric traffic is absorbed or filtered before reaching the application.
- Note / trade-off: A Kestrel/application limiter is not sufficient DDoS protection.

DownstreamServiceKey

- Definition: A separate secret proving that a proxied request reaching FinWallet.Api or a fake provider came through Gateway.
- FinWallet usage: A YARP transform adds it and the destination validates it through Shared.Web.
- Why / problem solved: It prevents direct backend-port access from automatically becoming trusted.
- Note / trade-off: It is intentionally distinct from InternalServiceKey to preserve separate trust boundaries.

Fail-Closed

- Definition: A security posture where an operation is denied when a required decision/control service fails.
- FinWallet usage: Protected transfer/purchase does not proceed when the external fraud provider is unavailable and returns 503.
- Why / problem solved: It prevents money movement without required risk evaluation.
- Note / trade-off: Availability can decrease; which dependencies fail closed depends on business risk appetite.

Fail-Open

- Definition: A posture where an operation proceeds even if a control dependency fails.
- FinWallet usage: FinWallet deliberately avoids it for fraud decisions; however post-commit communication failure does not roll money back.
- Why / problem solved: It can preserve availability for non-critical dependencies.
- Note / trade-off: For fraud/auth controls it can create financial risk, so the distinction must be explicit.

Fixed-Time Comparison

- Definition: Comparing secrets without early exit to reduce timing side-channel information.
- FinWallet usage: FinWallet validates internal service keys with CryptographicOperations.FixedTimeEquals.
- Why / problem solved: It makes it harder to infer correct secret prefixes from response timing.
- Note / trade-off: Length must still be handled and it does not eliminate every possible side channel.

HMAC

- Definition: A keyed mechanism for producing message-integrity/authentication values.
- FinWallet usage: FinWallet uses HMAC concepts for HS256 JWT signing and OTP protection.
- Why / problem solved: It prevents valid values from being forged without the secret key.
- Note / trade-off: HMAC is not encryption; it authenticates integrity but does not hide the message.

HS256 / HMAC-SHA256

- Definition: A symmetric JWT signing algorithm based on HMAC with SHA-256 and a shared secret.
- FinWallet usage: FinWallet uses it as a fixed access-token algorithm and requires a signing key of at least 32 UTF-8 bytes.

- Why / problem solved: It is simple and fast for a single trust domain.
- Note / trade-off: Anyone with the secret can sign tokens; asymmetric keys may be preferable across broader trust domains.

HSTS

- Definition: A browser policy instructing clients to use HTTPS only for a period of time.
- FinWallet usage: FinWallet can enable it through production configuration.
- Why / problem solved: It reduces HTTPS downgrade and accidental HTTP usage.
- Note / trade-off: Local development and TLS termination topology must be considered when enabling it.

InternalServiceKey

- Definition: A secret proving that FinWallet.Api is a trusted caller of Gateway internal/provider routes.
- FinWallet usage: It is sent as X-Internal-Service-Key and validated by the Gateway policy.
- Why / problem solved: It prevents normal clients from directly using provider routes.
- Note / trade-off: In production it must be injected from a secret store/environment and be sufficiently long.

JWT (JSON Web Token)

- Definition: A signed token format carrying identity claims.
- FinWallet usage: Login issues an access token; Gateway and FinWallet.Api validate issuer, audience, signature and lifetime.
- Why / problem solved: It provides authenticated identity on each request without storing the whole identity in process memory.
- Note / trade-off: JWT revocation is difficult by itself, so FinWallet combines it with durable sid session state.

Least Privilege

- Definition: The principle of granting only the minimum permissions required.
- FinWallet usage: FinWallet separates internal route credentials/policies, limits workflow permissions and avoids public backend ports.
- Why / problem solved: It reduces blast radius if credentials are compromised.
- Note / trade-off: Permissions that are too restrictive can break operations, so required capabilities should be explicit.

Logout / Session Revocation

- Definition: Making an active server-side session invalid.
- FinWallet usage: POST /api/v1/auth/logout revokes the sid carried by the JWT.
- Why / problem solved: Even before token expiry, API session checks can reject use of the token.
- Note / trade-off: The effectiveness depends on protected flows actually validating durable session state.

OTP (One-Time Password)

- Definition: A short-lived one-time verification code.

- FinWallet usage: FinWallet sends it by SMS for registration verification using TTL-based transient state.
- Why / problem solved: It provides an additional proof such as phone possession.
- Note / trade-off: An OTP is not an access token and must not be leaked in logs or normal responses.

OWASP API Security Top 10

- Definition: OWASP guidance focused on API risks such as BOLA, broken authentication, resource consumption and unsafe API consumption.
- FinWallet usage: FinWallet references it for gateway/API auth, owner-aware SQL, rate limits and provider ACL design.
- Why / problem solved: It separates API threats from traditional browser/UI risks.
- Note / trade-off: The list is not sufficient by itself; business abuse and fraud need separate controls.

OWASP Top 10

- Definition: OWASP guidance classifying common web-application security risks.
- FinWallet usage: FinWallet security documentation maps controls to access control, misconfiguration, cryptography, injection and logging concerns.
- Why / problem solved: It provides a shared security-review vocabulary/checklist.
- Note / trade-off: It is not a compliance certificate and does not replace threat modeling or real testing.

PBKDF2

- Definition: A password key-derivation algorithm intentionally made expensive against brute force.
- FinWallet usage: FinWallet stores credentials using PBKDF2 V1 with a fixed work factor and salt.
- Why / problem solved: It avoids plaintext passwords and weak fast hashes.
- Note / trade-off: The work factor cannot be changed casually through config; versioned migration/re-hash is required.

Pepper

- Definition: A secret application value added to hashing/HMAC and stored separately from the database.
- FinWallet usage: FinWallet can use a secret such as REGISTRATION_OTP_PEPPER for registration OTP protection.
- Why / problem solved: A database compromise alone is less useful without the separate secret.
- Note / trade-off: Pepper rotation needs an operational plan and the value must never be committed to source control.

PII (Personally Identifiable Information)

- Definition: Data that can directly or indirectly identify a real person.
- FinWallet usage: Phone/email are stored where needed for registration, while fraud/reconciliation responses avoid unnecessary PII.
- Why / problem solved: It supports data minimization in logs and internal APIs.
- Note / trade-off: PII definitions vary by regulation; production systems need formal privacy classification.

Refresh Token

- Definition: A longer-lived opaque credential used to obtain a new access token after expiry.
- FinWallet usage: FinWallet manages refresh state durably with rotation.
- Why / problem solved: It allows short access-token lifetimes without forcing frequent logins.
- Note / trade-off: It is high-value if stolen, so secure storage, rotation and revocation are required.

Refresh Token Rotation

- Definition: Replacing the refresh token on every successful refresh and invalidating the old one.
- FinWallet usage: FinWallet uses rotation to reduce refresh-token replay risk.
- Why / problem solved: It limits reuse of a previously stolen token.
- Note / trade-off: Concurrent refresh races must be handled with durable transaction/uniqueness rules.

Salt

- Definition: A random value added per password/credential before deriving the password hash.
- FinWallet usage: FinWallet PBKDF2 records use unique salts.
- Why / problem solved: It prevents equal passwords from producing equal hashes and weakens precomputed rainbow tables.
- Note / trade-off: A salt does not need to be secret; it must be unique and sufficiently random.

Secret Injection

- Definition: Providing secrets such as signing keys, DB passwords or service keys from deployment infrastructure instead of source code.
- FinWallet usage: Production appsettings uses empty placeholders and expects environment/secret-store overrides.
- Why / problem solved: It prevents secrets from being embedded in Git history or images.
- Note / trade-off: A local .env file is a development convenience, not a production secret manager.

Security Headers

- Definition: HTTP response headers that push browsers/clients toward safer defaults.
- FinWallet usage: Shared.Web centrally applies X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, CSP and no-store controls.
- Why / problem solved: It prevents individual endpoints from forgetting baseline headers.
- Note / trade-off: These headers do not replace authentication or authorization; they are hardening controls.

Service-to-Service Authentication

- Definition: Authentication where one backend service proves its identity to another.
- FinWallet usage: FinWallet.Api -> Gateway uses InternalServiceKey while Gateway -> backend/provider uses DownstreamServiceKey.
- Why / problem solved: It avoids reusing public user JWTs as machine trust credentials.
- Note / trade-off: Production systems may later prefer mTLS or managed identity over static service keys.

Session

- Definition: A server-side record representing an authenticated login lifecycle.

- FinWallet usage: sid, device and revoke/refresh information are stored durably in MSSQL.
- Why / problem solved: It provides logout, revocation and risk-state control even though JWT itself is stateless.
- Note / trade-off: Session state is not financial truth, but it is authoritative for authentication lifecycle.

SQL Injection

- Definition: A vulnerability where user input becomes part of SQL syntax and changes query behavior.
- FinWallet usage: FinWallet uses parameterized SQL and does not accept user-controlled SQL fragments.
- Why / problem solved: It reduces data exfiltration, modification and privilege-abuse risk.
- Note / trade-off: Parameters do not make dynamic table/column identifiers safe; those must remain server-owned.

SSRF (Server-Side Request Forgery)

- Definition: A vulnerability where client input makes the server request unintended internal or external URLs.
- FinWallet usage: FinWallet provider base URLs are server-owned configuration; clients cannot supply arbitrary destinations.
- Why / problem solved: It reduces risks such as metadata-service or internal-network access.
- Note / trade-off: Compromised configuration is a separate risk; network egress policy is still useful.

WAF (Web Application Firewall)

- Definition: An edge security layer that filters HTTP/L7 attack patterns.
- FinWallet usage: FinWallet does not implement a WAF in application code; it is recommended as production edge protection.
- Why / problem solved: It can block certain abuse/signature traffic before it reaches the application.
- Note / trade-off: It does not replace business authorization or fraud controls.

4. Data, Persistence and Concurrency (36 terms)

ACID

- Definition: A model summarizing transaction properties: Atomicity, Consistency, Isolation and Durability.
- FinWallet usage: FinWallet financial postings rely on MSSQL transaction guarantees.
- Why / problem solved: It is foundational for preventing partial commits and protecting rules under concurrency.
- Note / trade-off: External HTTP is not part of the ACID transaction; distributed consistency is handled separately.

Atomicity

- Definition: The property that all changes in a transaction either commit together or none commit.
- FinWallet usage: For transfers, source debit, destination credit, FinancialTransaction, Ledger and Idempotency commit or roll back together.
- Why / problem solved: It prevents one-sided money movement.

- Note / trade-off: External provider calls are outside this atomicity, so compensation/reconciliation are required.

Check Constraint

- Definition: A database constraint requiring row values to satisfy a logical condition.
- FinWallet usage: FinWallet uses schema-level checks for suitable amount/status/type/financial invariants.
- Why / problem solved: It reduces invalid rows even if application code has a bug.
- Note / trade-off: It is used for simple invariants rather than encoding the entire business workflow in SQL.

Commit

- Definition: Making transaction changes durable.
- FinWallet usage: Financial atomic posting is committed before a completed success result is returned.
- Why / problem solved: It defines the ordering between durable state and client success.
- Note / trade-off: A communication failure after commit does not roll money back; Outbox retry handles it.

Connection Pooling

- Definition: Reusing physical DB or HTTP connections from a pool instead of creating them for every operation.
- FinWallet usage: SqlConnection logical open/close uses provider pooling and HttpClient uses handler connection pools.
- Why / problem solved: It reduces handshake/login cost and latency.
- Note / trade-off: Bad pool sizing can overload the database or cause request queuing.

Consistency (Database)

- Definition: The property that data remains valid against constraints/invariants before and after a transaction.
- FinWallet usage: Ledger balancing, non-negative wallet rules and foreign/unique constraints support it.
- Why / problem solved: It reduces the chance of invalid persisted state.
- Note / trade-off: Business consistency requires both DB constraints and Application/Domain rules.

Database Migration

- Definition: A controlled change of database schema from one version to another.
- FinWallet usage: Docker mssql-init applies scripts such as 001, 002, 003 and 004 in order using SchemaVersions.
- Why / problem solved: It makes required schema reproducible for new code.
- Note / trade-off: Migrations need idempotent execution checks and a rollback or forward-fix strategy.

Database Transaction

- Definition: A mechanism executing a group of SQL changes as one commit/rollback unit.
- FinWallet usage: Wallet postings, corrections, outbox claiming/finalization and some auth changes use transactions.

- Why / problem solved: It keeps related rows changing together.
- Note / trade-off: External HTTP calls are deliberately kept outside the transaction.

Decimal Precision

- Definition: Storing monetary values with fixed decimal precision rather than binary floating point.
- FinWallet usage: Money amounts use decimal types in application and database.
- Why / problem solved: It prevents floating-point rounding errors from entering financial calculations.
- Note / trade-off: Scale and rounding rules must remain consistent domain invariants.

Durability

- Definition: The property that a committed transaction survives crashes.
- FinWallet usage: Completed transactions and ledger records are considered durable after MSSQL commit.
- Why / problem solved: It ensures financial state does not disappear after a success response.
- Note / trade-off: HA, backups and disaster recovery complement database durability at infrastructure level.

Durable State

- Definition: Persisted state that must survive process or container restarts.
- FinWallet usage: Wallet balances, ledger, transactions, idempotency, sessions, fraud reviews and outbox/inbox are durable in MSSQL.
- Why / problem solved: It preserves correctness and replay behavior after crashes or restarts.
- Note / trade-off: Durability alone does not guarantee correctness; transactions and constraints are also required.

Filtered Index

- Definition: A SQL Server index containing only rows that match a filter predicate.
- FinWallet usage: It can support small subsets such as pending/active rows.
- Why / problem solved: It provides targeted query performance with a smaller index.
- Note / trade-off: Required SQL Server SET options such as QUOTED_IDENTIFIER must be correct during migration.

Foreign Key

- Definition: A constraint requiring a reference in one table to point to an existing row in another.
- FinWallet usage: It protects relationships such as transaction details, ledger entries and wallet/customer links.
- Why / problem solved: It reduces orphaned data.
- Note / trade-off: Constraints add some write cost but are usually valuable for financial integrity.

GUID / UUID

- Definition: A 128-bit identifier format commonly used for uniqueness in distributed systems.
- FinWallet usage: Customer, Session, Wallet, Transaction, Journal and FraudEvent identities use GUIDs.
- Why / problem solved: It enables identifier creation without a central sequence coordinator.

- Note / trade-off: Random GUIDs can fragment clustered indexes, so physical index design still matters.

Index

- Definition: A data structure that helps queries locate rows faster.
- FinWallet usage: FinWallet uses indexes on appropriate customer, transaction, idempotency and history keys.
- Why / problem solved: Indexes can reduce logical reads and latency.
- Note / trade-off: Every index adds write/storage cost, so the project avoids speculative indexing without Query Store/plan evidence.

Isolation

- Definition: The transaction property controlling how concurrent operations observe each other's intermediate state.
- FinWallet usage: Financial posting stores use strong isolation/locking to prevent overspend and idempotency races.
- Why / problem solved: It helps stop concurrent debits from exceeding the same wallet balance.
- Note / trade-off: Strong isolation can increase lock contention and must be measured on hot paths.

Keyset Pagination

- Definition: A pagination method using the last-seen key as a cursor to fetch the next page.
- FinWallet usage: Transaction history uses a cursor such as beforeTransactionId with newest-first ordering.
- Why / problem solved: It avoids large OFFSET scans and is more stable with concurrent inserts.
- Note / trade-off: Jumping directly to arbitrary page numbers is harder; the client uses cursor-based navigation.

Lock

- Definition: A database concurrency mechanism preventing incompatible concurrent changes to the same data.
- FinWallet usage: SQL locking is part of wallet debit and idempotency correctness.
- Why / problem solved: It prevents two requests from reading the same balance and both spending it.
- Note / trade-off: Long transactions and inconsistent lock ordering increase deadlock risk.

MSSQL / SQL Server

- Definition: A relational database management system.
- FinWallet usage: FinWallet uses SQL Server as the durable source of truth for Customer, Session, Wallet, BankAccount, FinancialTransaction, Ledger, Idempotency, FraudEvent, Outbox/Inbox and Reconciliation.
- Why / problem solved: ACID transactions, constraints, locking and rich querying support financial correctness.
- Note / trade-off: As scale grows it needs indexing/partitioning/HA planning; it is not used as a transient cache substitute.

Optimistic Concurrency

- Definition: A concurrency approach that assumes conflicts are rare and detects them during update using versions/CAS semantics.
- FinWallet usage: FinWallet may use uniqueness/compare-and-set semantics in some paths while wallet posting also uses strong locking.
- Why / problem solved: It can reduce lock duration.
- Note / trade-off: For hot financial balances with frequent conflicts, pessimistic locking may be more suitable.

Parameterized SQL

- Definition: Sending SQL values as parameters instead of concatenating them into query text.
- FinWallet usage: FinWallet stores parameterize customer IDs, amounts, statuses and other values.
- Why / problem solved: It reduces SQL injection risk and can improve plan reuse.
- Note / trade-off: Table/column identifiers cannot be parameterized and must remain server-owned.

Persistence

- Definition: Storing and retrieving application state from a durable data store.
- FinWallet usage: Infrastructure SQL stores persist domain/application state to MSSQL.
- Why / problem solved: It provides lifecycle independent of process memory.
- Note / trade-off: The persistence model should not force Domain to depend on table structure.

Pessimistic Concurrency

- Definition: A concurrency approach that assumes conflicts are likely and locks relevant data during the operation.
- FinWallet usage: Critical wallet balance posting controls concurrent debits on the same rows.
- Why / problem solved: It directly prevents overspend inside the database transaction.
- Note / trade-off: Throughput can be limited by lock contention.

Race Condition

- Definition: A bug where correctness depends on the timing/order of concurrent operations.
- FinWallet usage: Parallel 600+600 spending from a 1000 wallet, duplicate idempotency keys or refresh-token races are examples.
- Why / problem solved: Financial correctness must remain deterministic across multiple instances.
- Note / trade-off: Single-thread unit tests are insufficient; real database concurrency tests are required.

RDB Snapshot

- Definition: A point-in-time disk snapshot of Redis memory state.
- FinWallet usage: The Docker runbook can request one using BGSAVE.
- Why / problem solved: It provides an additional local recovery/debug mechanism.
- Note / trade-off: Because it is point-in-time, it may miss recent writes and is never used instead of the financial ledger.

Redis

- Definition: A memory-first key/value store suited to TTLs, fast counters and transient coordination.

- FinWallet usage: FinWallet uses Redis for OTP TTLs, temporary counters, fraud velocity and hot/transient support state.
- Why / problem solved: It separates low-latency transient state from MSSQL workload.
- Note / trade-off: It is not authoritative for wallet balances or ledger data; losing Redis must not lose financial truth.

Redis AOF

- Definition: A Redis persistence mode that logs write commands to an append-only file for recovery after restart.
- FinWallet usage: Docker Redis can enable AOF to make transient support state more resilient locally.
- Why / problem solved: It reduces loss of some transient state across container restarts.
- Note / trade-off: Redis still does not become financial truth and AOF is not a substitute for MSSQL backups.

Rollback

- Definition: Discarding transaction changes before they are committed.
- FinWallet usage: Financial postings roll back on ledger imbalance, insufficient balance or SQL errors.
- Why / problem solved: It prevents partial database state.
- Note / trade-off: It cannot undo an already-completed external provider movement; compensation is required for that.

Schema

- Definition: The structural definition of database tables, columns, constraints, indexes and relationships.
- FinWallet usage: FinWallet versioned schema scripts create auth and financial tables.
- Why / problem solved: It supports data correctness with DB-level rules independent of application code.
- Note / trade-off: Schema changes require backward-compatibility and migration planning.

SchemaVersions

- Definition: A table/pattern recording which database migrations have already been applied.
- FinWallet usage: The init job checks it before running each migration script.
- Why / problem solved: It prevents accidental re-execution after container restarts.
- Note / trade-off: If an already-applied migration file is edited, the version record cannot detect that; applied migrations should be immutable.

Serializable Isolation

- Definition: One of the strongest standard SQL isolation levels, making concurrent effects close to serial execution.
- FinWallet usage: It can be used on critical atomic financial posting paths.
- Why / problem solved: It strongly limits overspend and phantom-style races.
- Note / trade-off: It increases lock contention/deadlock risk and should be limited to boundaries that need it.

Source of Truth

- Definition: The authoritative system considered correct for a piece of data.
- FinWallet usage: MSSQL is authoritative for FinWallet financial state; the bank provider is authoritative for its own external bank movements.
- Why / problem solved: It defines which system wins when caches or snapshots disagree.
- Note / trade-off: Multiple competing sources of truth create ambiguous ownership and reconciliation.

Transient State

- Definition: Short-lived state whose loss must not corrupt financial truth.
- FinWallet usage: OTP TTLs, fraud velocity counters and cache-like data may live in Redis.
- Why / problem solved: It provides fast access and reduces unnecessary durable-DB load.
- Note / trade-off: It must be reconstructable or safely expirable.

TTL (Time To Live)

- Definition: The duration after which a cache/key automatically expires.
- FinWallet usage: Registration OTP and some transient Redis state use TTLs.
- Why / problem solved: It prevents temporary verification data from living forever.
- Note / trade-off: TTL is not a retention policy for financial transactions; durable financial data is not expired this way.

Unique Constraint

- Definition: A database rule preventing duplicate values for a column or column combination.
- FinWallet usage: FinWallet uses uniqueness for idempotency, Inbox Source+MessageId and domain-specific keys.
- Why / problem solved: It provides a single winner even when multiple instances race to create the same row.
- Note / trade-off: It is safer than relying only on an application-side SELECT then INSERT check.

UTC / DateTimeOffset

- Definition: Representing timestamps in a timezone-neutral or offset-aware way.
- FinWallet usage: CreatedAt, CompletedAt and ReviewedAt timestamps use UTC/DateTimeOffset semantics.
- Why / problem solved: It improves ordering and audit consistency across services and regions.
- Note / trade-off: Local business cutoff rules still require country/timezone context.

5. Reliability, Messaging and Distributed Systems (22 terms)

At-Least-Once Delivery

- Definition: A messaging guarantee that aims for at least one delivery and allows duplicates.
- FinWallet usage: The Outbox worker retries failures, so a message can have multiple delivery attempts.
- Why / problem solved: It accepts duplicate risk instead of silently losing messages.
- Note / trade-off: Exactly-once-like behavior requires idempotent consumers or deduplication.

Background Worker / Hosted Service

- Definition: A process component that performs continuous or periodic work outside an HTTP request.
- FinWallet usage: BankMoneyMovementBackgroundService and OutboxDispatchBackgroundService serve this role.
- Why / problem solved: They progress long-running/pending provider flows without tying them to client request lifetime.
- Note / trade-off: Worker state must live in durable storage rather than process memory for restart safety.

Backoff

- Definition: Delaying retries, often progressively, so a failing dependency is not hammered continuously.
- FinWallet usage: Outbox failure handling can schedule the next attempt for a later time.
- Why / problem solved: It reduces retry storms during dependency outages.
- Note / trade-off: Backoff alone does not replace retry limits or dead-letter/manual handling decisions.

Callback

- Definition: A server-to-server call where an external provider later reports operation status.
- FinWallet usage: The internal bank callback endpoint processes Pending/Completed/Failed provider state through the Inbox.
- Why / problem solved: It avoids depending solely on client polling for long-running provider operations.
- Note / trade-off: Callbacks require authentication, deduplication and replay-safe finalization.

Claim / Lease

- Definition: Temporarily assigning work to one worker so other workers do not process it concurrently.
- FinWallet usage: The Outbox dispatcher atomically claims messages in SQL.
- Why / problem solved: It reduces concurrent duplicate sends across multiple worker instances.
- Note / trade-off: If the worker crashes after claiming, the item must eventually become claimable again.

Compensating Transaction

- Definition: A new transaction that offsets a prior financial transaction instead of deleting it.
- FinWallet usage: Refund and Reversal create an opposite journal/transaction without mutating the original.
- Why / problem solved: It preserves auditability and immutable history.
- Note / trade-off: Not every transaction type uses the same correction path; external-bank operations may require provider compensation.

Compensation

- Definition: A business action that offsets a previously completed effect in a distributed flow.
- FinWallet usage: Examples include releasing blocked balance after provider failure or applying a corrective flow for external/local mismatch.
- Why / problem solved: It handles cases where a database rollback cannot undo a completed external action.
- Note / trade-off: Compensation is not rollback; it is a new auditable business action.

Deduplication

- Definition: Preventing a repeated logical event/request/message from applying its effect again.
- FinWallet usage: Inbox unique keys, idempotency keys and terminal-state checks provide deduplication at different layers.
- Why / problem solved: It makes network retries and provider duplicates safe.
- Note / trade-off: A bad dedupe key can incorrectly collapse distinct real operations.

Eventual Consistency

- Definition: A model where different systems are allowed to become consistent after a delay rather than instantly.
- FinWallet usage: Money can commit in MSSQL before a notification is later sent via Outbox; callbacks and reconciliation also arrive asynchronously.
- Why / problem solved: It enables reliable integration without a distributed transaction.
- Note / trade-off: The system must explicitly define which state is strongly consistent and which is eventually consistent.

Exactly-Once Effect

- Definition: The goal that a business effect occurs once even if transport delivers a message multiple times.
- FinWallet usage: FinWallet approaches this with idempotency, Inbox and terminal-state guards rather than claiming exactly-once transport.
- Why / problem solved: It provides practical duplicate-safe financial behavior.
- Note / trade-off: The system does not claim absolute network-level exactly-once delivery.

ExternalTransactionId

- Definition: A transaction identifier generated by the external bank/provider.
- FinWallet usage: It links local FinancialTransaction records to provider status queries, callbacks and reconciliation.
- Why / problem solved: It separates FinWallet transaction identity from provider transaction identity.
- Note / trade-off: It is not a Correlation ID; it is the provider's business-operation identity.

Idempotency

- Definition: The property that repeating the same logical request does not apply its financial effect more than once.
- FinWallet usage: Transfer, Purchase, BankDeposit/Withdrawal and correction commands use durable Idempotency-Key handling.
- Why / problem solved: It prevents duplicate money movement under timeout, retry and network ambiguity.
- Note / trade-off: Same key + same payload replays the result; same key + different payload must conflict.

Idempotency-Key

- Definition: A stable request key supplied by the client so the server can recognize a repeated financial command.

- FinWallet usage: It is required as a header on money-moving public POST endpoints.
- Why / problem solved: It allows client retries to replay an existing result instead of creating another transaction.
- Note / trade-off: It is distinct from Correlation ID and carries duplicate-protection semantics.

Inbox Pattern

- Definition: A pattern that durably records incoming messages/callbacks and deduplicates them by a stable identity.
- FinWallet usage: FakeBank callbacks are deduplicated using Source + MessageId in the Inbox.
- Why / problem solved: It prevents repeated provider callbacks from applying financial finalization multiple times.
- Note / trade-off: The underlying business operation should also be replay-safe because Inbox alone cannot eliminate every crash window.

Non-Retryable Error

- Definition: A terminal/business error where retrying the same request is not meaningful.
- FinWallet usage: Invalid account, permanent provider rejection or insufficient funds can fail the operation and release blocked funds.
- Why / problem solved: It prevents endless retries and permanently stuck blocked balances.
- Note / trade-off: Incorrect classification can either reduce availability or create duplicate-effect risk.

Outbox Pattern

- Definition: A reliability pattern that stores an outgoing message in the same database transaction as business state and sends it later via a worker.
- FinWallet usage: Purchase/Bank-movement completion can create notification Outbox rows that a worker sends to FakeCommunication.
- Why / problem solved: It prevents a committed money operation from permanently losing its notification after a crash.
- Note / trade-off: It typically provides at-least-once delivery, so downstream effects should be idempotent.

Provider Idempotency Key

- Definition: A stable key sent by FinWallet to an external provider to prevent duplicate downstream side effects.
- FinWallet usage: Bank-account opening and bank-movement requests carry provider request keys.
- Why / problem solved: It provides downstream duplicate protection independently of client idempotency.
- Note / trade-off: If the provider does not guarantee key semantics, FinWallet does not blindly retry.

Replay

- Definition: Returning the result of an already-completed idempotent operation.
- FinWallet usage: Transfer/Purchase handlers check completed replay state before calling fraud/providers again.
- Why / problem solved: It prevents duplicate requests from repeating money movement or external-service cost.

- Note / trade-off: Replay responses should preserve original transaction identity and completion time.

Request Hash

- Definition: A hash of a canonical request payload used with an idempotency key to prove the payload is unchanged.
- FinWallet usage: Fraud/idempotency flows can hash canonical source/destination/amount data with SHA-256.
- Why / problem solved: It detects reuse of the same key with a different payload.
- Note / trade-off: Canonicalization must be deterministic so formatting differences do not create false conflicts.

Retry

- Definition: Trying an operation again after a transient failure.
- FinWallet usage: Outbox communication and retryable bank-status processing can be retried in a controlled way.
- Why / problem solved: It improves availability under transient provider/network failures.
- Note / trade-off: Financial POSTs are never blindly retried unless provider idempotency semantics make that safe.

Retryable Error

- Definition: A transient error that may succeed if the same operation is attempted later.
- FinWallet usage: Provider timeout/network interruption or temporary communication outage are examples.
- Why / problem solved: It allows workers to remain pending and retry with backoff.
- Note / trade-off: Business rejection or insufficient funds are not retryable; error classification must be accurate.

Terminal State

- Definition: A final transaction state that requires no further normal lifecycle processing.
- FinWallet usage: Completed and certain Failed/Denied states are terminal.
- Why / problem solved: Terminal checks prevent duplicate callbacks/retries from moving money again.
- Note / trade-off: Manual corrections may create a new child transaction while leaving the original terminal state unchanged.

6. Finance, Accounting and Banking (57 terms)

Asset

- Definition: An accounting account type representing economic resources owned or controlled by the company.
- FinWallet usage: BANK-SETTLEMENT:TRY represents the banking-side backing asset in the FinWallet ledger.
- Why / problem solved: It makes the backing for wallet liabilities visible in the accounting equation.
- Note / trade-off: The current FakeBank model does not fully model a separate real FinWallet omnibus account; settlement asset is currently an accounting abstraction.

Atomic Posting

- Definition: Applying all financial posting components in one database transaction as all-or-nothing.
- FinWallet usage: Balance, FinancialTransaction, Journal/Entries, idempotency and when needed Outbox commit together.
- Why / problem solved: It prevents partial money movement and ledger/balance divergence.
- Note / trade-off: External providers are outside this atomic posting boundary.

Available Balance

- Definition: The portion of wallet balance immediately available for spending or transfer.
- FinWallet usage: Transfers and purchases debit the source wallet's available balance.
- Why / problem solved: It is central to overspend prevention.
- Note / trade-off: Pending external withdrawals may move funds from available to blocked.

Bank Account

- Definition: An external account representing the customer's real/provider-side bank account.
- FinWallet usage: FinWallet links a BankAccount record to a wallet using FakeBank externalAccountId, IBAN, currency and status.
- Why / problem solved: It separates bank-held money from FinWallet wallet balance and provides the boundary for funding/withdrawal.
- Note / trade-off: BankAccount balance and Wallet balance do not have to be equal.

Bank Reference

- Definition: An external reference associated with a bank/provider operation.
- FinWallet usage: It can be used with ExternalTransactionId for transaction history and troubleshooting.
- Why / problem solved: It makes matching the same movement across systems easier for support and reconciliation.
- Note / trade-off: It must not be confused with the internal FinWallet transaction ID.

Bank Settlement Asset

- Definition: An asset-account abstraction representing FinWallet's bank-side backing for customer funds.
- FinWallet usage: It is debited on Bank->Wallet funding and credited down when Wallet->Bank withdrawal settles.
- Why / problem solved: It lets the ledger track the banking-side backing of customer wallet liabilities.
- Note / trade-off: Because FakeBank does not yet model a distinct FinWallet omnibus/safeguarding account, this is not a one-to-one record of a real provider account.

Bank Statement

- Definition: A chronological list/statement of completed movements on an external bank account.
- FinWallet usage: IBankProvider.GetStatementAsync retrieves provider statement items for reconciliation.
- Why / problem solved: It allows comparison between FinWallet local bank-movement records and provider truth.
- Note / trade-off: Statement retrieval is read-only and does not mutate financial state.

Bank-Settlement Reconciliation

- Definition: Comparing internal bank-transaction records with their expected settlement-ledger effects.
- FinWallet usage: Completed BankDeposit/Withdrawal amount/status is checked against bank-settlement entries.
- Why / problem solved: It detects divergence between local transaction state and accounting state.
- Note / trade-off: It is distinct from external provider-statement reconciliation.

BankDeposit

- Definition: From FinWallet's perspective, funding the digital wallet from the customer's external bank account.
- FinWallet usage: The client calls POST /bank-movements/deposits; the provider adapter maps it to a FakeBank Withdrawal from the customer bank account, then local ledger increases settlement asset and wallet liability.
- Why / problem solved: It converts bank-held funds into spendable FinWallet balance.
- Note / trade-off: Do not confuse direction names: FinWallet Deposit means money enters the wallet; at FakeBank the same economic movement is a withdrawal from the customer account.

BankWithdrawal

- Definition: From FinWallet's perspective, moving money from the digital wallet back to the external bank account.
- FinWallet usage: Funds can move from available to blocked, cutoff/provider processing occurs, and completion decreases wallet liability and settlement asset while crediting the customer's bank account.
- Why / problem solved: It prevents spending the same funds while an external bank movement is pending.
- Note / trade-off: Provider failure requires correct blocked-fund release/compensation.

Blocked Balance

- Definition: Wallet funds temporarily reserved and unavailable but not yet finally settled out of the wallet.
- FinWallet usage: During a pending Wallet->Bank withdrawal, amount moves from available to blocked.
- Why / problem solved: It prevents the same money from being spent again in another transfer or purchase.
- Note / trade-off: A terminal provider failure must release the blocked amount or funds become stuck.

Business Day / Business Calendar

- Definition: A calendar defining working days and holidays for banking operations.
- FinWallet usage: FakeCutoff simulates weekend/holiday/processing-date rules.
- Why / problem solved: It correctly models the difference between calendar days and bank processing days.
- Note / trade-off: Production requires an authoritative holiday-calendar source and timezone management.

Campaign

- Definition: A business rule applying a discount under merchant/customer/amount conditions.

- FinWallet usage: FakeCampaign returns eligibility, discount amount and sponsor type; FinWallet performs the accounting.
- Why / problem solved: It separates promotion logic from financial posting.
- Note / trade-off: The provider only returns a decision/calculation and never writes the ledger.

Campaign Expense

- Definition: An expense account representing a discount funded by the platform.
- FinWallet usage: For a 200 TRY purchase with a 20 TRY platform discount: debit customer liability 180, debit campaign expense 20, credit merchant payable 200.
- Why / problem solved: It makes the economic source of the discount explicit in the ledger.
- Note / trade-off: A merchant-sponsored discount may not use the same expense account; merchant payable can be net of the discount.

Credit

- Definition: The right-side entry direction in double-entry accounting.
- FinWallet usage: It can increase a liability or decrease an asset; BankDeposit credits the customer wallet liability.
- Why / problem solved: Together with debit it keeps the journal balanced.
- Note / trade-off: Credit does not always mean 'money arrived'; interpretation depends on account type.

Currency

- Definition: The code/enum identifying the monetary unit.
- FinWallet usage: TRY, USD and EUR create currency boundaries for wallets and accounts.
- Why / problem solved: It prevents accidental cross-currency posting.
- Note / trade-off: V1 has no FX conversion; different currency wallets are not automatically converted.

Customer Wallet Liability

- Definition: A ledger liability account representing the customer's wallet balance as money FinWallet owes the customer.
- FinWallet usage: It increases with a credit on BankDeposit and decreases with a debit on source transfer or purchase.
- Why / problem solved: It captures the correct accounting meaning of wallet balance.
- Note / trade-off: The Wallet current balance should reconcile to the corresponding ledger liability.

Cutoff

- Definition: A time/rule boundary affecting whether a bank operation processes on the same or a later business day.
- FinWallet usage: FakeCutoff decides using the business calendar and transaction type.
- Why / problem solved: It allows after-hours bank withdrawals to become Scheduled rather than failing.
- Note / trade-off: Cutoff is not necessarily one universal time; it can vary by country, currency and operation type.

Debit

- Definition: The left-side entry direction in double-entry accounting.
- FinWallet usage: It can increase an asset or decrease a liability; BankDeposit debits settlement asset, while a transfer debits the source wallet liability to reduce it.
- Why / problem solved: It models economic direction according to account type.
- Note / trade-off: Debit does not always mean 'money went out'.

Digital Wallet

- Definition: An internal financial account representing a customer's spendable digital-money balance in FinWallet.
- FinWallet usage: Each wallet is currency-specific and has AvailableBalance/BlockedBalance plus a ledger liability relationship.
- Why / problem solved: It supports fast internal transfers, purchases and refunds independently of the external bank.
- Note / trade-off: It is not the same as a BankAccount; wallet balance represents FinWallet's obligation to the customer.

Discount

- Definition: The campaign amount deducted from a purchase's original price.
- FinWallet usage: OriginalAmount and DiscountAmount can be stored in transaction details/history.
- Why / problem solved: It separates the customer's net payment from the merchant's economic entitlement.
- Note / trade-off: Ledger entries depend on who sponsors the discount.

Double-Entry Bookkeeping

- Definition: An accounting method where every event affects at least two accounts and total debit equals total credit.
- FinWallet usage: FinWallet uses it for transfers, bank deposits, purchases and corrections.
- Why / problem solved: The balance equation helps detect money being created or disappearing without an accounting source.
- Note / trade-off: Debit/credit do not mean good/bad or simply plus/minus; their effect depends on account type.

Expense

- Definition: An accounting account type representing a cost incurred by the company.
- FinWallet usage: A platform-sponsored campaign discount can debit a CAMPAIGN-EXPENSE-type account.
- Why / problem solved: It makes the economic sponsor explicit instead of silently reducing merchant or customer value.
- Note / trade-off: The accounting posting depends on who sponsors the campaign.

External Bank Statement Reconciliation

- Definition: Comparing FinWallet local bank movements with FakeBank/real-bank statements.
- FinWallet usage: ExternalTransactionId, amount, status and references are used to detect mismatches.

- Why / problem solved: It finds movements completed at the provider but missing locally, or the reverse.
- Note / trade-off: Provider I/O occurs outside the SQL transaction and results are later persisted as reconciliation data.

FinancialTransaction

- Definition: A durable business-transaction record carrying lifecycle, amount, type, status and references.
- FinWallet usage: Transfer, BankDeposit, BankWithdrawal, Purchase, Refund and Reversal are tracked through this model.
- Why / problem solved: It separates business transaction identity/lifecycle from accounting journals.
- Note / trade-off: FinancialTransaction is not the ledger itself: the transaction says what happened; the ledger says the accounting effect.

FinancialTransactionStatus

- Definition: The lifecycle state of a FinancialTransaction.
- FinWallet usage: States such as Scheduled, Pending, Completed and Failed are used in financial/bank flows.
- Why / problem solved: Workers, callbacks and history APIs use it to understand current progress.
- Note / trade-off: Terminal states are important guards against duplicate posting.

FinancialTransactionType

- Definition: The enum/type identifying which business movement a FinancialTransaction represents.
- FinWallet usage: Values include WalletTransfer, BankDeposit, BankWithdrawal, Purchase, Refund and Reversal.
- Why / problem solved: It lets posting, correction and history logic understand transaction semantics.
- Note / trade-off: Database numeric contracts and code enum values must remain aligned.

IBAN

- Definition: An international standardized identifier for a bank account.
- FinWallet usage: FakeBank generates an example IBAN when an external account is opened and FinWallet stores it on BankAccount.
- Why / problem solved: It provides a familiar external account reference.
- Note / trade-off: The simulator IBAN is not connected to a real banking clearing network.

Ledger

- Definition: The accounting book that immutably records all FinWallet financial movements as debit/credit entries across accounts.
- FinWallet usage: Every completed financial posting creates a LedgerJournal with at least two LedgerEntries and total debits must equal total credits.
- Why / problem solved: It answers not only 'what is the balance now?' but also 'how was this balance formed?' in an auditable way.
- Note / trade-off: The Wallet table is current state/projection; Ledger is accounting history. Original entries are not deleted; corrections use opposite postings.

Ledger Account

- Definition: An accounting account that receives debit/credit entries in the ledger.
- FinWallet usage: Logical accounts can include BANK-SETTLEMENT:TRY, WALLET-LIABILITY:<walletId> and MERCHANT-PAYABLE:<merchant>.
- Why / problem solved: It identifies the economic account affected by each movement.
- Note / trade-off: A Ledger Account is not the same concept as the customer's BankAccount entity.

Ledger Entry

- Definition: A single debit or credit line inside a LedgerJournal.
- FinWallet usage: It carries account, side (Debit/Credit), amount/currency and references.
- Why / problem solved: It decomposes a business event into atomic accounting movements.
- Note / trade-off: An entry alone does not describe the entire transaction; it is interpreted as part of its journal.

Ledger Journal

- Definition: A record grouping all debit/credit entries belonging to one financial event.
- FinWallet usage: A transfer, deposit, purchase or correction creates a journal whose entries must balance.
- Why / problem solved: It allows the accounting impact of one business event to be audited as a unit.
- Note / trade-off: Journals are not mutated; reversals/refunds create new journals.

Liability

- Definition: An accounting account type representing an obligation owed to another party.
- FinWallet usage: A customer's wallet balance is modeled as WALLET-LIABILITY:<wallet>, an obligation FinWallet owes the customer.
- Why / problem solved: It prevents treating customer wallet funds as FinWallet's own money.
- Note / trade-off: Liabilities normally increase with credits and decrease with debits.

Merchant

- Definition: The commercial party from whom the customer purchases goods or services.
- FinWallet usage: Purchase requests carry merchantId and Campaign/Fraud evaluation can use merchant context.
- Why / problem solved: It identifies the counterparty and merchant payable account for spending.
- Note / trade-off: V1 is not a complete merchant onboarding/settlement platform.

Merchant Payable

- Definition: A liability account representing the amount FinWallet owes a merchant after a purchase.
- FinWallet usage: The purchase journal credits merchant payable according to the campaign sponsorship model.
- Why / problem solved: It converts customer wallet debit into a merchant settlement obligation.
- Note / trade-off: Actual bank settlement of merchant payable can be a separate operational scope.

Money

- Definition: A financial value object combining Amount and Currency.
- FinWallet usage: Transfer, purchase, bank-movement and fraud-evaluation amounts use Money semantics.
- Why / problem solved: It prevents 100 TRY and 100 USD from being treated as the same value.
- Note / trade-off: Rounding and scale rules belong to the Money invariant.

Negative Balance

- Definition: A wallet or related financial balance falling below zero contrary to business rules.
- FinWallet usage: Negative customer-wallet balances/overspend are forbidden v1 invariants.
- Why / problem solved: Without a credit/overdraft product, this keeps financial correctness simple.
- Note / trade-off: A future credit product would need an explicit limit/loan model rather than silently weakening the invariant.

Omnibus Account

- Definition: A bank account model where a fintech/payment institution holds funds belonging to many customers in one pooled account.
- FinWallet usage: FinWallet does not yet fully model a separate real FakeBank omnibus account; the BANK-SETTLEMENT ledger asset represents the economic idea in accounting.
- Why / problem solved: It explains how one physical bank balance can be allocated to many customer wallet liabilities through the internal ledger.
- Note / trade-off: Production regulations may require safeguarding, segregation and legal-ownership controls.

Original Transaction

- Definition: The original completed financial transaction referenced by a Refund or Reversal.
- FinWallet usage: Correction checks use original type, status and ownership through parent transaction references.
- Why / problem solved: It makes the economic event being corrected auditable.
- Note / trade-off: The original row/history is not deleted or overwritten.

Overspend

- Definition: Spending or transferring more than the wallet's available balance.
- FinWallet usage: Atomic SQL balance checks/locking prevent it, including under concurrent requests.
- Why / problem solved: It prevents customers from spending nonexistent value and protects ledger/balance solvency.
- Note / trade-off: A simple application-side if(balance >= amount) is not race-safe.

Parent Transaction

- Definition: A reference showing which previous transaction a correction/child transaction was derived from.
- FinWallet usage: Refund/Reversal history can expose ParentTransactionId.
- Why / problem solved: It makes the original-to-compensating transaction chain visible.
- Note / trade-off: Parent linkage does not replace accounting entries; each journal must still balance.

Payable

- Definition: A liability representing an amount the company owes to a merchant or third party.
- FinWallet usage: A purchase increases MERCHANT-PAYABLE:<merchant> with a credit.
- Why / problem solved: It converts customer spending into a settlement obligation owed to the merchant.
- Note / trade-off: Actual bank transfer to the merchant may be a separate settlement process.

Posting

- Definition: Persisting the financial effect of a business event into wallet state and the double-entry ledger.
- FinWallet usage: Transfer posting writes source/destination balances, transaction, journal, entries and idempotency result.
- Why / problem solved: It converts a business decision into durable accounting state.
- Note / trade-off: Fraud/provider external I/O is not performed while the posting transaction is open.

Processing Date

- Definition: The business date on which a bank operation is processed.
- FinWallet usage: FakeCutoff can derive it from country, currency, transaction type and request time.
- Why / problem solved: It separates request timestamp from banking business-day processing.
- Note / trade-off: It is not the same as a UTC timestamp; it carries business-calendar semantics.

Provider Deposit / Withdrawal Direction

- Definition: Directions defined from the FakeBank account's perspective.
- FinWallet usage: FakeBank Deposit increases the bank account and Withdrawal decreases it. A FinWallet BankDeposit may therefore invoke the opposite provider direction.
- Why / problem solved: It clarifies that the word 'deposit' depends on system perspective.
- Note / trade-off: The adapter/ACL isolates this semantic difference so provider enums do not leak into Domain.

Purchase

- Definition: A financial transaction where the customer uses wallet balance to pay a merchant.
- FinWallet usage: After Fraud and Campaign evaluation, wallet liability decreases, merchant payable increases and sponsorship may add an expense/netting entry.
- Why / problem solved: It converts wallet spending into an obligation owed to the merchant.
- Note / trade-off: The external bank account does not need to move for every purchase; this is an internal ledger-based model.

Reconciliation

- Definition: The process of comparing expected and observed financial records across two or more sources and reporting differences.
- FinWallet usage: FinWallet has Wallet<->Ledger, BankTransaction<->SettlementLedger and FinWallet<->FakeBankStatement scopes.
- Why / problem solved: It detects silent drift, bugs, missing callbacks and provider mismatches.

- Note / trade-off: FinWallet reconciliation never silently changes balances; it records issues without mutating financial history.

Reconciliation Issue

- Definition: A durable record representing a mismatch discovered during reconciliation.
- FinWallet usage: It can store expected/actual amounts, transaction/wallet/bank references and non-PII details.
- Why / problem solved: It makes the discrepancy visible and traceable.
- Note / trade-off: Resolution may require manual investigation/correction; the system does not silently overwrite data.

Reconciliation Run

- Definition: A single reconciliation execution for a defined scope.
- FinWallet usage: RunId, scope, status, timestamps and issue count are stored durably.
- Why / problem solved: It lets operations audit when and what reconciliation was performed.
- Note / trade-off: A reconciliation run does not automatically imply a correction.

Refund

- Definition: A correction that returns all or a defined portion of a completed Purchase to the customer.
- FinWallet usage: FinWallet v1 performs full purchase refund by creating a new FinancialTransaction and opposite journal without deleting the original.
- Why / problem solved: It restores customer wallet liability and reverses merchant/campaign effects while preserving audit history.
- Note / trade-off: Refund does not mutate the original purchase and is linked through parent transaction identity.

Reversal

- Definition: A new transaction that reverses the economic effect of a completed transaction.
- FinWallet usage: Public reversal safely handles internal WalletTransfer by moving value destination->source with an opposite ledger journal.
- Why / problem solved: It corrects money movement while preserving immutable history.
- Note / trade-off: External BankDeposit/Withdrawal is not reversed through this endpoint; provider compensation is required.

Safeguarding

- Definition: The practice of protecting customer funds by separating them legally/operationally from the company's own operating funds.
- FinWallet usage: FinWallet v1 does not implement a full regulatory safeguarding model; settlement-asset/liability separation provides the conceptual accounting basis.
- Why / problem solved: It makes clear that wallet balances are not company revenue and must be backed/protected.
- Note / trade-off: A real product requires jurisdiction-specific regulation, licensing, bank-partner and legal-accounting design.

Settlement

- Definition: The process of finally discharging the economic obligation between parties in a financial transaction.
- FinWallet usage: Bank-movement processing/settlement dates and BANK-SETTLEMENT ledger accounts use this concept.
- Why / problem solved: It separates internal book entries from real external-bank movement.
- Note / trade-off: Internal posting and external settlement do not have to occur at the same instant, so reconciliation is needed.

Settlement Date

- Definition: The business date on which a bank/financial operation is expected to reach final settlement.
- FinWallet usage: The Cutoff provider returns processing/settlement dates that can be stored in transaction details.
- Why / problem solved: It explains when a scheduled/pending operation is expected to settle.
- Note / trade-off: Actual provider callback time can differ from the settlement date; reconciliation is still required.

Sponsor

- Definition: The party economically funding a campaign discount.
- FinWallet usage: Posting differs between platform-sponsored and merchant-sponsored discounts.
- Why / problem solved: It determines which accounting account bears the discount.
- Note / trade-off: Sponsor is not just a UI label; it is an accounting input.

Wallet Transfer

- Definition: An internal same-currency movement from one FinWallet wallet to another.
- FinWallet usage: After fraud approval, source liability is debited, destination liability credited and balances updated atomically.
- Why / problem solved: It provides internal book transfer without a bank call.
- Note / trade-off: Source/destination currency, overspend and idempotency rules must hold.

Wallet-Ledger Reconciliation

- Definition: Comparing the current Wallet table balance with the economic balance derivable from the ledger.
- FinWallet usage: It is one of the reconciliation scopes.
- Why / problem solved: It detects projection drift or incorrect posting.
- Note / trade-off: A mismatch does not cause automatic wallet overwrite; an issue is created.

7. Fraud and Risk (17 terms)

Allow

- Definition: The fraud decision allowing the operation to continue.
- FinWallet usage: When durable fraud state is Allow/Approved, transfer/purchase proceeds to atomic posting.
- Why / problem solved: It lets the business flow continue after risk checks.

- Note / trade-off: Allow does not prove absence of fraud; it means risk is acceptable under current signals.

Deny

- Definition: The fraud decision preventing the operation from being executed.
- FinWallet usage: Internal fraud can deny directly or the combined policy can produce a final Deny.
- Why / problem solved: It prevents risky operations from reaching wallet/ledger posting.
- Note / trade-off: Reason codes should be safely recorded for audit/operations without exposing sensitive model details to clients.

External Fraud Provider

- Definition: An integration boundary obtaining part of the fraud decision from an external service.
- FinWallet usage: FakeFraud simulates Allow/Review/Deny, score and reason codes.
- Why / problem solved: It keeps FinWallet's internal fraud logic independent from an external risk engine.
- Note / trade-off: Protected flows fail closed when the provider is unavailable.

Fraud

- Definition: The problem domain of detecting and preventing unauthorized, malicious or risky financial behavior.
- FinWallet usage: FinWallet evaluates internal and external fraud before WalletTransfer and Purchase.
- Why / problem solved: It places risk decisions before money movement.
- Note / trade-off: Current v1 BankDeposit does not call fraud; documentation intentionally does not invent such a step.

Fraud Decision

- Definition: The Allow, Review or Deny outcome of a fraud assessment.
- FinWallet usage: Internal and external decisions are combined into a final decision by policy.
- Why / problem solved: It explicitly determines whether financial posting can proceed.
- Note / trade-off: The decision is not the same as HTTP status; for example Review can map to 202.

Fraud Decision Policy

- Definition: A business policy combining internal and external fraud decisions into one final decision.
- FinWallet usage: For example, any Deny may produce final Deny, Review combinations can produce Review, and only safe combinations Allow.
- Why / problem solved: It avoids scattering decision-merging conditionals through handlers.
- Note / trade-off: Changing the policy changes risk appetite and requires testing/audit.

Fraud Score

- Definition: A numeric provider/model output representing risk level.
- FinWallet usage: FakeFraud can return a score; FinWallet does not blindly equate score with the final decision.
- Why / problem solved: It provides an input for threshold/policy decisions.
- Note / trade-off: Score calibration is model/provider-specific and can change over time.

FraudEvent

- Definition: A durable record of fraud evaluation identity, internal/external/final decision, reason codes and review state.
- FinWallet usage: WalletTransfer/Purchase use it to replay fraud state for the same idempotent request.
- Why / problem solved: It prevents repeated provider calls and lost decisions while supporting manual-review audit.
- Note / trade-off: Raw sensitive payloads/hashes/PII are not unnecessarily exposed through internal responses.

Internal Fraud Engine

- Definition: FinWallet's deterministic/rule-based risk component using server-side signals.
- FinWallet usage: It evaluates amount, velocity, new-device and beneficiary/merchant familiarity signals.
- Why / problem solved: It catches obvious risks before external provider calls and adds domain-specific controls.
- Note / trade-off: It is not a complete production ML fraud system and requires real-data tuning.

Known Beneficiary

- Definition: A signal indicating whether the transfer destination is previously known to the customer.
- FinWallet usage: It can be used in WalletTransfer risk context.
- Why / problem solved: It helps distinguish risky combinations such as large transfers to a new beneficiary.
- Note / trade-off: Familiarity must be derived server-side from history rather than a client-supplied boolean.

Manual Fraud Review

- Definition: A pending FraudEvent being approved or denied by an internal operator/reviewer.
- FinWallet usage: /api/v1/internal/fraud-reviews runs behind the Gateway InternalService policy.
- Why / problem solved: It provides human-in-the-loop risk decisions without introducing a normal customer/admin user type.
- Note / trade-off: Reviewer identity and review timestamp must be audited and the decision should be final once made.

Merchant Familiarity

- Definition: A risk signal indicating whether the purchase merchant is familiar from customer history.
- FinWallet usage: Purchase fraud can use merchant familiarity instead of beneficiary familiarity.
- Why / problem solved: It helps assess patterns such as a new merchant combined with a high amount.
- Note / trade-off: It is not proof of fraud by itself and is combined with other signals.

New Device

- Definition: A risk signal indicating a device reference not previously seen for the session/customer.
- FinWallet usage: Fraud logic can treat a new-device transaction as higher risk.

- Why / problem solved: It helps expose activity after credential theft from a different device.
- Note / trade-off: Device fingerprints have privacy/spoofing limitations and should not be the sole deny reason.

Reason Code

- Definition: A short machine-readable code explaining why a fraud or business decision was made.
- FinWallet usage: Internal/external fraud reason codes are normalized and stored on FraudEvent.
- Why / problem solved: It supports operations, analytics and testing without parsing free-form text.
- Note / trade-off: Clients should not receive enough rule detail to facilitate model/rule bypass.

Review

- Definition: A fraud decision requiring manual review instead of automatic allow or deny.
- FinWallet usage: It creates a durable pending FraudEvent and the public request can return 202 Accepted.
- Why / problem solved: It pauses suspicious-but-not-certainly-invalid operations before money movement.
- Note / trade-off: Review state is durable; replaying the same idempotent request should not call external fraud again.

Risk Signal

- Definition: Server-side behavioral/context data used in fraud assessment.
- FinWallet usage: Examples include transaction count, 24-hour amount, country, device, known beneficiary and merchant familiarity.
- Why / problem solved: It builds risk context on the server rather than trusting clients to declare themselves safe.
- Note / trade-off: Poor signal quality increases false positives and false negatives.

Velocity

- Definition: A fraud signal measuring transaction frequency or amount intensity over a time window.
- FinWallet usage: FinWallet can use transaction count over five minutes and amount over 24 hours.
- Why / problem solved: It helps detect rapid account takeover or automated abuse patterns.
- Note / trade-off: Redis counters can be transient; they are not financial truth.

8. Performance and Observability (13 terms)

Correlation / Traceability

- Definition: The ability to connect logs, provider calls and database records belonging to one business flow.
- FinWallet usage: CorrelationId, TransactionId, ExternalTransactionId, FraudReference and Outbox/Inbox message IDs are used together.
- Why / problem solved: It simplifies finding where a distributed flow failed.
- Note / trade-off: One universal ID should not be overloaded with every semantic meaning.

Health Endpoint

- Definition: An HTTP endpoint used by orchestrators, proxies and operators to check service health.
- FinWallet usage: Docker smoke tests and YARP health mechanisms use /health/* endpoints.
- Why / problem solved: It provides an availability signal for startup and routing automation.
- Note / trade-off: Health endpoints should not expose sensitive dependency or secret details.

Latency

- Definition: The elapsed time from the start of an operation/request to its result.
- FinWallet usage: Gateway, provider HTTP, MSSQL and Redis all contribute to total API latency.
- Why / problem solved: It is a primary metric for user experience and timeout design.
- Note / trade-off: Average latency is insufficient; p95/p99 tail latency should also be observed.

Liveness

- Definition: A health concept indicating whether the process itself is alive and not irrecoverably stuck.
- FinWallet usage: Gateway/API live endpoints can signal whether a container should be restarted.
- Why / problem solved: It avoids restarting a healthy process merely because a dependency is temporarily unavailable.
- Note / trade-off: Separating it from readiness improves production orchestration.

Log Rotation

- Definition: Rotating logs by size/count so they do not grow without bound.
- FinWallet usage: Docker json-file logging is configured with rotation limits.
- Why / problem solved: It reduces the risk of host/service failure due to full disks.
- Note / trade-off: It is not a substitute for centralized log retention; it is local disk safety.

Logical Reads

- Definition: A measure of how many database pages a SQL query reads from the buffer cache.
- FinWallet usage: FinWallet considers it alongside elapsed time for query/index tuning.
- Why / problem solved: It reveals unnecessary scans even on a fast test machine.
- Note / trade-off: It should be interpreted with CPU, waits, row counts and execution plans.

Observability

- Definition: The ability to understand internal system behavior through logs, metrics, traces and health signals.
- FinWallet usage: FinWallet provides a foundation with correlation IDs, structured logs, health endpoints and durable transaction/reconciliation records.
- Why / problem solved: It helps answer why incidents or performance changes occurred.
- Note / trade-off: A full production setup still needs centralized log, metrics and tracing backends.

p95 / p99

- Definition: Latency percentiles below which 95% or 99% of requests complete.
- FinWallet usage: Performance review uses them for hot endpoints, providers and SQL paths.
- Why / problem solved: They expose slow-tail behavior hidden by averages.

- Note / trade-off: Percentiles require sufficient samples and an appropriate time window.

Query Plan

- Definition: The SQL Server execution plan describing indexes, joins and access methods used for a query.
- FinWallet usage: FinWallet performance tuning checks plans and logical reads before adding indexes.
- Why / problem solved: It supports evidence-based tuning instead of guesswork.
- Note / trade-off: Plans vary with data distribution and environment; one small test database is not representative.

Query Store

- Definition: A SQL Server feature collecting query history, plans and runtime statistics.
- FinWallet usage: It is recommended for finding regressions and expensive queries in production-like environments.
- Why / problem solved: It makes the effect of query/index changes measurable.
- Note / trade-off: Storage/retention settings require management and Query Store does not replace application logging.

Readiness

- Definition: A health concept indicating whether a service is ready to receive new traffic.
- FinWallet usage: It can prevent API traffic before MSSQL/Redis/schema initialization is ready.
- Why / problem solved: It reduces startup races and dependency-related request failures.
- Note / trade-off: Not every optional dependency should necessarily make the entire service unready.

Structured Logging

- Definition: Logging using named fields rather than only free-form message text.
- FinWallet usage: CorrelationId, TransactionId, path, status and safe error codes can be logged as structured fields.
- Why / problem solved: It improves search, aggregation and incident investigation.
- Note / trade-off: Passwords, OTPs, tokens, connection strings and unnecessary PII must not be logged as structured fields.

Throughput

- Definition: The number of requests or transactions a system can process over a period.
- FinWallet usage: Gateway, API, SQL pools and provider capacity all bound FinWallet throughput.
- Why / problem solved: It is used for scaling and capacity planning.
- Note / trade-off: Throughput is never increased by weakening financial correctness, locking or idempotency invariants.

9. Testing and Quality (15 terms)

Chaos Test

- Definition: A test that deliberately injects failures such as dependency delays, outages or restarts to validate recovery behavior.

- FinWallet usage: Fake-provider fail/delay/timeout modes, container restarts and communication outages provide the basis.
- Why / problem solved: It validates fail-closed behavior, Outbox retry, blocked-fund release and reconciliation.
- Note / trade-off: Production chaos testing requires controlled blast radius and strong observability.

Concurrency Test

- Definition: A test verifying that concurrent requests on the same resource do not violate invariants.
- FinWallet usage: For example, with a 1000 balance only one of two parallel 600 transfers should complete.
- Why / problem solved: It proves overspend, idempotency-race and locking behavior on the real database.
- Note / trade-off: It should use real parallel requests and durable-state assertions rather than artificial sleeps.

Definition of Done

- Definition: The set of criteria required before a feature is considered complete.
- FinWallet usage: FinWallet requires code plus affected API, DB, security, tests, docs, configuration and runtime validation to be updated.
- Why / problem solved: It reduces cases where code is merged while tests or documentation remain stale.
- Note / trade-off: The checklist is a living standard and can evolve as the project grows.

End-to-End (E2E) Test

- Definition: A test covering a critical flow from the client entry point to final durable outcome using real components.
- FinWallet usage: Register -> login -> wallet -> bank movement -> transfer/purchase paths can be E2E tests.
- Why / problem solved: It validates that the system actually works from the user's perspective.
- Note / trade-off: Failures are harder to localize than unit tests, so E2E suites should focus on critical scenarios.

Fixture

- Definition: Pre-arranged data or environment state used by a test.
- FinWallet usage: Integration/E2E tests may seed customers, wallets, bank accounts and balances as fixtures.
- Why / problem solved: Fixtures provide repeatable scenarios.
- Note / trade-off: Financial fixtures should preserve ledger consistency instead of directly editing wallet balances.

Integration Test

- Definition: A test exercising multiple real components/dependencies together.
- FinWallet usage: FinWallet targets Docker-based integration across Gateway, FinWallet.Api, MSSQL, Redis and fake providers.
- Why / problem solved: It catches DI, configuration, serialization, schema and network-boundary failures.

- Note / trade-off: It is slower and more environment-dependent than unit testing.

Load Test

- Definition: A test that applies controlled high traffic to measure throughput, latency and resource behavior.
- FinWallet usage: It is needed to validate Gateway/API/SQL/Redis pool and rate-limit values before production.
- Why / problem solved: It makes configuration tuning evidence-based.
- Note / trade-off: Financial test data and side effects must remain isolated in a safe environment.

Mock

- Definition: A test double whose behavior is controlled by the test instead of using the real dependency.
- FinWallet usage: Moq can mock IBankProvider, stores or external dependencies in unit tests.
- Why / problem solved: It makes failure and edge-case behavior fast and deterministic.
- Note / trade-off: Mocks do not prove real provider/SQL behavior and excessive mocking can couple tests to implementation details.

Moq

- Definition: A .NET mocking library used to create interface/class test doubles.
- FinWallet usage: FinWallet tests use it to isolate external/store dependencies.
- Why / problem solved: Setup/Verify makes expected-interaction testing straightforward.
- Note / trade-off: It is not a substitute for real integration testing.

Smoke Test

- Definition: A quick post-build/deploy check that the system starts and key dependencies are reachable.
- FinWallet usage: Docker CI starts all services and validates Gateway health, MSSQL schema and Redis connectivity.
- Why / problem solved: It catches runtime DI/config/startup issues that static build validation misses.
- Note / trade-off: It is not a deep business-correctness test.

Strict Mock

- Definition: A mock mode that fails when an unconfigured dependency call occurs.
- FinWallet usage: Some handler tests use Moq Strict to catch unexpected provider/store calls.
- Why / problem solved: It can prove, for example, that the bank provider is never called after an early validation failure.
- Note / trade-off: Overly strict tests can become refactor-fragile, so it is best used for meaningful business interactions.

Test Double

- Definition: A general term for a fake/mock/stub-like object replacing a real dependency in tests.
- FinWallet usage: FinWallet uses test doubles to control provider/store behavior in unit tests.
- Why / problem solved: They make tests fast and deterministic.

- Note / trade-off: The chosen double style should keep the test's intent clear.

Unit Test

- Definition: A test focusing on one class or small business behavior with external dependencies isolated.
- FinWallet usage: FinWallet.Application.Tests validates handler behavior using mocks/test doubles.
- Why / problem solved: It provides fast deterministic feedback for branches and error handling.
- Note / trade-off: It does not prove real SQL locking, Redis atomicity or Gateway routing.

Warnings as Errors

- Definition: Treating compiler warnings as build failures.
- FinWallet usage: FinWallet Release CI builds with --warnaserror.
- Why / problem solved: It prevents new warnings from accumulating silently.
- Note / trade-off: Third-party or generated warnings may require targeted, justified suppression.

xUnit

- Definition: A .NET unit/integration testing framework.
- FinWallet usage: FinWallet.Application.Tests uses xUnit v3.
- Why / problem solved: It provides fact/theory-based testing and CI integration.
- Note / trade-off: A test framework is not a testing strategy by itself; scenario selection and coverage still require design.

10. Docker, CI/CD and Configuration (25 terms)

.env

- Definition: A local file holding environment-variable values for Docker Compose/development.
- FinWallet usage: .env.example is versioned while the real .env is gitignored.
- Why / problem solved: It makes local configuration easy to bootstrap.
- Note / trade-off: It is not a production secret-management mechanism.

appsettings.json

- Definition: .NET's base JSON configuration file.
- FinWallet usage: FinWallet defines platform limits, provider URLs/timeouts, Redis/SQL tuning, Swagger and Gateway configuration here.
- Why / problem solved: It separates operational configuration from code.
- Note / trade-off: Financial and cryptographic invariants are not exposed as casual runtime switches.

appsettings.Production.json

- Definition: The standard .NET configuration file overriding base settings in the Production environment.
- FinWallet usage: It can disable Swagger by default and provide production-oriented settings with empty secret placeholders.
- Why / problem solved: It separates development and production behavior without hardcoding environment branches.

- Note / trade-off: Actual secrets are not committed and must be injected externally.

BuildKit Cache

- Definition: A Docker build cache mechanism that reuses layers/restore output across builds.
- FinWallet usage: FinWallet .NET images use NuGet restore cache mounts and can lock sharing to avoid parallel cache corruption.
- Why / problem solved: It reduces CI build time.
- Note / trade-off: Cache is never more important than correctness; clean builds remain useful when cache corruption is suspected.

CI (Continuous Integration)

- Definition: The practice of automatically building/testing/validating changes before they reach the main branch.
- FinWallet usage: FinWallet CI runs Release builds with warnings-as-errors and tests; Docker workflows validate runtime behavior.
- Why / problem solved: It catches compile, runtime and schema regressions early.
- Note / trade-off: Green CI does not guarantee all production correctness; critical E2E, security and load testing still matter.

Compose Overlay

- Definition: An additional YAML file that overrides the base Compose configuration for a specific environment.
- FinWallet usage: compose.debug.yml exposes localhost debug ports and compose.production.yml adds production-like overrides.
- Why / problem solved: It allows environment variation without cluttering one file with every conditional.
- Note / trade-off: Overlay order matters and incorrect combinations can expose unintended ports/settings.

Configuration Precedence

- Definition: The order determining which value wins when the same configuration key appears in multiple sources.
- FinWallet usage: FinWallet follows the usual appsettings -> environment-specific appsettings -> environment variables/secret injection layering.
- Why / problem solved: It enables the same image to be configured per environment.
- Note / trade-off: Resolved configuration can contain secrets and should not be logged.

Container

- Definition: A running instance of an image with isolated process, filesystem and networking.
- FinWallet usage: Each FinWallet HTTP service plus MSSQL/Redis runs in its own container.
- Why / problem solved: It provides dependency/version isolation and fast recreation.
- Note / trade-off: Application containers are designed stateless; durable data lives in volumes/databases.

Docker

- Definition: A platform for running applications and dependencies as isolated container images/runtimes.
- FinWallet usage: Gateway, FinWallet.Api, fake providers, MSSQL and Redis can be started with Docker Compose.
- Why / problem solved: It makes local/integration environments reproducible.
- Note / trade-off: Containers do not automatically solve production orchestration or HA.

Docker Compose

- Definition: A tool for defining and running multiple container services, networks and volumes together.
- FinWallet usage: compose.yml defines the complete FinWallet stack.
- Why / problem solved: It provides one-command local full-stack startup, dependency ordering and configuration.
- Note / trade-off: It does not have to be the final production orchestrator.

Docker DNS / Service Discovery

- Definition: Automatic hostname resolution of Compose service names inside a Docker network.
- FinWallet usage: Gateway destinations can use names such as finwallet-api and fake-bank.
- Why / problem solved: It removes the need for hardcoded container IPs.
- Note / trade-off: Service names are environment-specific and should be managed through configuration.

Docker Network

- Definition: An isolated virtual network where containers can reach each other by service/DNS name.
- FinWallet usage: finwallet-backend connects HTTP services while finwallet-data connects FinWallet.Api to MSSQL/Redis.
- Why / problem solved: It provides connectivity without publishing databases to the host.
- Note / trade-off: Network segmentation does not replace credentials/authentication.

Dockerfile

- Definition: A declarative file describing how a container image is built.
- FinWallet usage: docker/Dockerfile.webapi provides the shared multi-stage build for Gateway/API/fake services.
- Why / problem solved: It makes build steps reproducible and version-controlled.
- Note / trade-off: Runtime images should remain small and non-root where practical.

Environment Variable

- Definition: A method of supplying runtime configuration through the process environment.
- FinWallet usage: Connection strings, passwords, JWT/service keys and tuning settings can override appsettings.
- Why / problem solved: It enables environment-specific configuration without changing the image/source.
- Note / trade-off: Secret environment variables still have exposure risks; production secret-store/orchestrator integration is preferable.

GitHub Actions

- Definition: GitHub's automation platform for running CI/workflows on repository events.
- FinWallet usage: FinWallet uses it for restore/build/test, Docker validation/smoke and PDF generation.
- Why / problem solved: It provides reproducible quality gates and artifacts for pull requests.
- Note / trade-off: Workflow permissions and secret access should follow least privilege.

Image

- Definition: An immutable filesystem/runtime package used to create containers.
- FinWallet usage: .NET service images are built through a multi-stage Dockerfile.
- Why / problem solved: It makes the same binary/runtime package consistent across environments.
- Note / trade-off: Secrets and mutable runtime data are not baked into the image.

Multi-Stage Build

- Definition: A technique separating build tooling and final runtime into different Docker stages.
- FinWallet usage: The SDK stage restores/builds/publishes while the final ASP.NET runtime stage contains only published output.
- Why / problem solved: It reduces final image size and attack surface.
- Note / trade-off: Poor cache/copy ordering can slow CI builds.

Named Volume

- Definition: Docker-managed persistent storage whose lifecycle is independent of an individual container.
- FinWallet usage: finwallet_mssql_data, finwallet_mssql_backup and finwallet_redis_data are named volumes.
- Why / problem solved: They preserve data across container recreation.
- Note / trade-off: docker compose down -v deletes them and is documented as destructive.

Non-Root Container

- Definition: Running the application process inside the container without root privileges.
- FinWallet usage: FinWallet .NET runtime images use a USER app style configuration.
- Why / problem solved: It reduces privilege impact if the process/container is compromised.
- Note / trade-off: Filesystem and port permissions must be planned accordingly.

Read-Only Filesystem

- Definition: A hardening approach where the application container root filesystem is not writable at runtime.
- FinWallet usage: Stateless FinWallet API containers can use a read-only root filesystem where practical.
- Why / problem solved: It reduces malware/config tampering and accidental local-state risks.
- Note / trade-off: Temporary/runtime write paths may need explicit tmpfs or volumes.

Resource Limit

- Definition: A cap on container CPU, memory or PID consumption.

- FinWallet usage: Compose can define local safety-baseline limits for FinWallet application containers.
- Why / problem solved: It limits one faulty service from consuming the entire host.
- Note / trade-off: Production limits should be based on load testing and capacity measurement.

SBOM (Software Bill of Materials)

- Definition: An inventory listing software components and versions contained in a build/artifact.
- FinWallet usage: FinWallet documentation considers it a supply-chain improvement rather than a core runtime component.
- Why / problem solved: It helps quickly identify affected artifacts during vulnerability incidents.
- Note / trade-off: Generating an SBOM does not remediate vulnerabilities by itself; scanning and remediation workflows are required.

Supply Chain Security

- Definition: Protecting the build/package/dependency chain from malicious or vulnerable components.
- FinWallet usage: Central NuGet versions, explicit dependencies, CI restore/build/test and planned vulnerability/SBOM controls belong here.
- Why / problem solved: It reduces risk originating from third-party components even when application code is correct.
- Note / trade-off: Pinning packages alone is not enough; provenance, vulnerability and update processes are needed.

Vulnerability Scan

- Definition: Automated checking of dependencies, container images or code for known security vulnerabilities.
- FinWallet usage: It can serve as a supply-chain quality gate in FinWallet CI hardening.
- Why / problem solved: It makes known-CVE risk visible before merge/deployment.
- Note / trade-off: Human review may still be needed for false positives and exploitability context.

Workflow Artifact

- Definition: A downloadable file package produced during a CI workflow run.
- FinWallet usage: Generated TR/EN PDF sets are also uploaded as GitHub Actions artifacts.
- Why / problem solved: It makes binary output easy to inspect/render before merge.
- Note / trade-off: Artifact retention is temporary; final versioned PDFs are also committed to the repository.

11. .NET and Application Coding (15 terms)

.NET 8

- Definition: The target runtime/framework version for FinWallet.
- FinWallet usage: All main APIs and fake-provider projects build on .NET 8.
- Why / problem solved: It provides modern ASP.NET Core, built-in DI, rate limiting, HttpClientFactory and container support.
- Note / trade-off: Framework upgrades require separate compatibility and testing work.

ASP.NET Core

- Definition: The .NET framework used to build HTTP APIs and web applications.
- FinWallet usage: FinWallet.Gateway, FinWallet.Api and simulator APIs use ASP.NET Core.
- Why / problem solved: It provides Kestrel, middleware, authentication, controllers, DI and configuration.
- Note / trade-off: The business domain is kept independent of framework APIs.

async/await

- Definition: .NET's asynchronous programming model for waiting on I/O without blocking a thread.
- FinWallet usage: HTTP, SQL, Redis and background-worker I/O paths are asynchronous.
- Why / problem solved: It reduces thread-pool starvation under load.
- Note / trade-off: Async does not automatically make work parallel or transactional.

Built-in DI Container

- Definition: ASP.NET Core's built-in dependency injection container.
- FinWallet usage: FinWallet uses IServiceCollection registration instead of a third-party DI container.
- Why / problem solved: It manages the dependency graph with standard framework mechanisms.
- Note / trade-off: No extra container is introduced without a real need for advanced interception or scoping features.

CancellationToken

- Definition: A .NET cancellation signal propagated through asynchronous operations.
- FinWallet usage: FinWallet passes it from controllers through handlers to SQL/Redis/HttpClient calls.
- Why / problem solved: It reduces unnecessary I/O after a client disconnects or cancels.
- Note / trade-off: It does not undo an already committed financial transaction; cancellation only stops work that has not durably committed.

Central Package Management

- Definition: Managing NuGet package versions from one central file instead of each csproj.
- FinWallet usage: A Directory.Packages.props-style setup pins xUnit, Moq, YARP, Swashbuckle and other versions centrally.
- Why / problem solved: It reduces version drift and duplicated version declarations across the solution.
- Note / trade-off: Major upgrades still require compatibility review; central pinning is not automatic security.

Controller-Based API

- Definition: Defining ASP.NET Core endpoints with Controller classes and attribute routing.
- FinWallet usage: FinWallet uses controllers instead of Minimal APIs.
- Why / problem solved: It keeps authorization, response metadata and organization explicit for a larger API surface.
- Note / trade-off: It can add more boilerplate for very small endpoints.

Exception Mapping

- Definition: Converting application/domain exceptions into controlled HTTP status and machine-code responses.
- FinWallet usage: Expected failures such as insufficient balance, fraud deny/review, idempotency conflict and provider outage are mapped to ServiceResult responses.
- Why / problem solved: It prevents business errors from surfacing as generic 500s.
- Note / trade-off: Unexpected stack traces are not exposed to clients; they are logged and mapped to a safe 500.

Immutable Model

- Definition: A model where key financial-history records are not modified after creation.
- FinWallet usage: Ledger journals/entries and completed history are corrected without overwriting originals.
- Why / problem solved: It preserves auditability and reasoning over historical events.
- Note / trade-off: Current-state projections such as Wallet balance can be mutable; the distinction between history and projection matters.

Machine-Readable Error Code

- Definition: A stable string code allowing clients/operations to identify an error without parsing free-form text.
- FinWallet usage: Examples include IDEMPOTENCY_CONFLICT, FRAUD_REVIEW_REQUIRED and RATE_LIMITED.
- Why / problem solved: Client behavior remains stable even if human-readable messages change or are localized.
- Note / trade-off: A code should not carry conflicting semantics across endpoints; this is why the error-code catalog exists.

Middleware

- Definition: A cross-cutting component that runs before/after controllers in the HTTP request pipeline.
- FinWallet usage: Shared.Web applies security headers, method/content checks, correlation and rate-limit behavior.
- Why / problem solved: It avoids repeating platform controls in every controller.
- Note / trade-off: Business use-case logic is not moved into middleware.

NuGet

- Definition: .NET's package/dependency distribution ecosystem.
- FinWallet usage: YARP, Swashbuckle, Moq, xUnit and SQL/Redis client libraries are obtained through NuGet.
- Why / problem solved: It simplifies reusable library/tooling integration.
- Note / trade-off: Package count, licensing and vulnerabilities require review because dependencies create supply-chain risk.

TimeProvider

- Definition: A .NET abstraction for obtaining time instead of calling DateTime.UtcNow directly.
- FinWallet usage: FinWallet uses it for Application logic such as fraud evaluation/review timestamps.
- Why / problem solved: It makes time-dependent tests deterministic.

- Note / trade-off: Differences between application time and DB/provider authoritative time still require consideration.

Typed HttpClient

- Definition: A .NET pattern combining HttpClient configuration and provider-specific calls in a strongly typed client class.
- FinWallet usage: Bank/Fraud/Cutoff/Campaign/Communication provider clients are registered as typed HttpClients.
- Why / problem solved: It keeps base URL, timeout, handler and provider API calls inside one boundary.
- Note / trade-off: The client class should not contain domain business orchestration.

XML Documentation Comments

- Definition: C# /// <summary> comments used for IDE, Swagger and code documentation.
- FinWallet usage: FinWallet uses a TR/EN documentation standard on public classes/interfaces/methods.
- Why / problem solved: It helps new engineers understand code intent quickly.
- Note / trade-off: Stale comments become misinformation, so documentation updates are part of Definition of Done.

12. Terms Deliberately Avoided or Replaced (11 terms)

ASP.NET Core Identity

- Definition: Microsoft's built-in framework for user, password, token, role and authentication persistence.
- FinWallet usage: FinWallet does not use it because the project implements custom auth/session/refresh behavior.
- Why / problem solved: This gives explicit control over banking-style sessions, refresh rotation, OTP and schema behavior.
- Note / trade-off: In production, a battle-tested identity solution can reduce risk; custom authentication carries more security responsibility.

Blind Retry

- Definition: Automatically retrying an operation without understanding its side effects or idempotency semantics.
- FinWallet usage: FinWallet avoids it for financial provider POSTs.
- Why / problem solved: If the first request succeeds at the provider but the response is lost, a blind retry can duplicate money movement.
- Note / trade-off: Retry becomes safe only with stable provider idempotency keys and explicit operation-status semantics.

CQRS

- Definition: An approach separating command/write models from query/read models.
- FinWallet usage: FinWallet does not implement a full CQRS framework, though write handlers and history read handlers are naturally separated.
- Why / problem solved: V1 prefers simple Application handlers instead of extra mediator/read-store complexity.

- Note / trade-off: CQRS may become useful later if read scaling or model divergence grows significantly.

Direct Balance UPDATE

- Definition: Manually changing a wallet balance column without a matching transaction/ledger entry.
- FinWallet usage: FinWallet documentation explicitly warns against it.
- Why / problem solved: It breaks the relationship between Wallet projection and Ledger, creating unexplained money and reconciliation mismatches.
- Note / trade-off: Even test funding should use a balanced provider/ledger fixture or an official financial posting flow.

Distributed Transaction

- Definition: The problem of coordinating multiple databases/services as if they committed atomically.
- FinWallet usage: FinWallet does not keep external HTTP inside MSSQL transactions or implement 2PC-style distributed transactions.
- Why / problem solved: It uses local ACID, idempotency, compensation and reconciliation to reduce locking and operational complexity.
- Note / trade-off: Explicit eventual consistency is accepted instead of synchronous cross-system atomicity.

Event Sourcing

- Definition: An architecture where domain events are the source of truth and current state is rebuilt from them.
- FinWallet usage: FinWallet does not use Event Sourcing; it has immutable double-entry Ledger history but the entire domain is not event-sourced.
- Why / problem solved: Ledger provides accounting auditability while Wallet current state and relational tables keep operations/querying practical.
- Note / trade-off: A ledger is not the same thing as Event Sourcing.

Generic Repository

- Definition: A generic repository abstraction exposing the same CRUD interface for every entity.
- FinWallet usage: FinWallet avoids it on financial paths and uses use-case-specific boundaries such as IWalletTransferPostingStore and IFraudEventStore.
- Why / problem solved: This prevents financial transaction, locking and idempotency semantics from being hidden behind generic CRUD.
- Note / trade-off: Generic abstractions can suit simple CRUD, but they were not chosen for critical financial paths.

MediatR

- Definition: A popular .NET mediator library for dispatching requests to handlers.
- FinWallet usage: FinWallet deliberately does not use it; controllers invoke handlers directly through DI.
- Why / problem solved: This keeps call paths visible and avoids an unnecessary abstraction/package layer.
- Note / trade-off: A mediator can be reconsidered if cross-cutting handler pipelines become genuinely complex.

Microservices

- Definition: A distributed architecture where services can be deployed and scaled independently.
- FinWallet usage: FinWallet deliberately avoids microservices for the v1 financial core while keeping fake external providers as separate HTTP services.
- Why / problem solved: Avoiding premature separation of Wallet/Ledger/Transaction reduces distributed-consistency cost for atomic money movement.
- Note / trade-off: Boundaries can be reconsidered if independent scaling or team ownership becomes a real requirement.

Redis as Financial Source of Truth

- Definition: Using Redis as the sole authoritative store for wallet/ledger financial state.
- FinWallet usage: FinWallet deliberately avoids this.
- Why / problem solved: MSSQL remains authoritative so Redis restart, eviction or operational behavior cannot define money correctness.
- Note / trade-off: Redis is a performance/transient-support layer, not financial truth or audit history.

Two-Phase Commit (2PC)

- Definition: A protocol coordinating multiple resource managers through prepare and commit phases.
- FinWallet usage: FinWallet v1 does not use it for bank/provider integrations.
- Why / problem solved: Local transaction patterns are preferred because providers typically do not support 2PC and the protocol adds availability/complexity costs.
- Note / trade-off: Most real HTTP banking providers are not distributed XA participants.